

CopernicusLAC Infrastructure Support

System Architecture Description

TERRADUE

Advancing Earth Science



T Systems



Title	Copernicus LAC Infra - D2 System Architecture Description
Prepared by	Terradue
Reference	T2-ESA-COPLACI-DD-24-029
Issue	1.3
Date	2025-07-27
Status	Issued
Document Type	Design Document
Configuration	Configured document
ESA Contract No:	4000144570/24/I-KE

Change record

Date	Issue	Section	Page	Comment
2024-08-15	1.0	All	All	Initial issue
2024-12-03	1.1	All 3.1 4.1, 4.2, 4.4	All 16 - 18 38 - 44	• Typos • Include different levels of ingestion, metadata registration, dataset copying, and dataset calibration
2025-05-28	1.2	All 2.2	All 10	• Typos • Roles naming alignment with D1
2025-07-27	1.3	2.5		• Updated Section 2.5 with PostgreSQL DBaaS indication, S3-compatible object storage-as-a-service requirement, and support for cold storage tier integration.

Table of Content

1 Introduction	5
1.1 Purpose	5
1.2 Background	5
1.3 Scope	6
1.4 Document Structure	6
1.5 Applicable Documents	7
1.6 Reference Documents	7
1.7 Acronyms and Abbreviations	7
2 System Overview	9
2.1 System Objectives	9
2.2 User Roles	9
2.3 System Components	10
2.4 High-Level Architecture Diagram	12
2.5 Assumptions and Constraints	14
2.6 Summary of Key Requirements	15
3 System Components	17
3.1 Data Ingestion	17
3.2 Data Discovery and Access	20
3.3 Data Processing	22
3.4 Application Registry	26
3.5 Front-end Services	28
3.6 User Workspace	31
3.7 Authentication and Authorization	34
3.8 Resource Management	36
4 Data Ingestion Process	39
4.1 Technologies Used	39
4.2 Ingestion Workflow	40
4.3 Application Package Selection	42
4.4 Data Validation and Content Validation	43
4.5 Metadata Profile	44
4.6 Metadata Enrichment for Visualization	45
5 Data Discovery, Access and Visualisation	48
5.1 SpatioTemporal Asset Catalog (STAC)	48
5.2 Catalogue API	50
5.3 Data Access and Retrieval	52
5.4 Graphical User Interface	54
5.5 Visualisation Integration	57
6 Application Packaging	59

6.1 Overview	59
6.2 Common Workflow Language (CWL)	60
6.3 Containers	62
6.4 Deployment	63
6.5 Version Control and Update Mechanisms	65
6.6 Access Control	66
6.7 Benefits	68
6.8 Compliance with Platform Standards	69
7 Technologies and Frameworks	72
7.1 Purpose	72
7.2 Common Workflow Language (CWL)	72
7.3 Docker	73
7.4 Kubernetes	73
7.5 GitLab	73
7.6 Harbor	74
7.7 Argo Workflows	74
7.8 Kafka	74
7.9 Crossplane	74
7.10 Prometheus	74
7.11 Grafana	75
7.12 ZOO Project	75
7.13 Titiler	75
7.14 PostgreSQL	76
8 Security and Compliance	77
8.1 Purpose	77
8.2 Security Framework	77
8.3 Data Protection and Privacy	77
8.4 Access Controls	78
8.5 Monitoring and Auditing	79
8.6 Incident Response	79
9 Scalability and Performance	81
9.1 Purpose	81
9.2 Scalability Strategies	81
9.3 Performance Optimization	81
9.4 Load Testing and Benchmarking	82
9.5 Resource Management	82

1 Introduction

1.1 Purpose

The purpose of this document is to provide the System Architecture Description for the platform of the *CopernicusLAC Centre* in the context of the *Copernicus LAC Infrastructure Support* project.

1.2 Background

The CopernicusLAC initiative is a collaborative effort between the European Space Agency (ESA), the European Commission's Directorate-General for International Partnerships (DG-INTPA), and the Government of Panama. The initiative aims to establish a Regional Copernicus Centre in Panama to facilitate the uptake and utilisation of Copernicus data in the LAC region. The primary focus of the Centre is to support disaster risk reduction and recovery efforts, leveraging Earth Observation (EO) data and geospatial technology to provide information services to local and regional stakeholders.

This initiative is part of the broader EU-LAC Digital Alliance, which promotes cooperation in digital and space technologies between the European Union (EU) and LAC countries. The CopernicusLAC Centre is envisioned as a pilot project under the EU's Global Gateway strategy, aiming to foster international collaboration, enhance data sharing, and support the development of local capacities in EO data processing and utilisation.

The CopernicusLAC Centre is a regional facility dedicated to the management, processing, and dissemination of EO data from the Copernicus Program, specifically tailored for the LAC region. The centre aims to provide comprehensive services for data hosting, processing, and access, supporting a wide range of applications, including environmental monitoring, disaster risk reduction, and sustainable development. The centre also focuses on enhancing local capacities by transferring knowledge and providing training to local operators.

The CopernicusLAC Centre Platform refers to the technical infrastructure and software environment that enables the functionalities of the CopernicusLAC Centre. This platform includes the hardware, software, middleware, and network components required to ingest, process, store, and distribute Copernicus EO data. The platform supports various workflows and tools necessary for data management and analysis, ensuring access and usability for different stakeholders. It is designed to be scalable and capable of handling large volumes of data and supporting a growing number of users and applications.

Based on stakeholder and user engagement, several EO-based Information will be co-designed focusing on 1) Risk and Recovery mapping, 2) Spatially explicit vulnerability and preparedness assessment for Disaster Risk Reduction on a national to regional level,

including baseline maps, and 3) EO monitoring of extreme weather and climate-related events (e.g. heatwaves, extreme rainfall, droughts, flooding and wildfires, etc.)

The Platform introduces tools and services, leveraging open EO data and geospatial technology, to perform the pre-operational demonstration of the Information Services and to deliver and transfer validated products to selected mandated stakeholders.

1.3 Scope

The scope of this document is to provide a comprehensive description of the system architecture for the CopernicusLAC Centre. It outlines the high-level design, components, and functionalities required to establish and operate the CopernicusLAC infrastructure, ensuring effective data hosting, exploration, processing, and federation.

This document serves as a blueprint for the technical implementation and operational setup of the CopernicusLAC Centre, guiding the development and integration of various system components. It aims to provide a clear understanding of the system's architecture to all stakeholders, including project managers, developers, and operators.

This document does not delve into the detailed technical specifications of interfaces, operations, and maintenance, which are covered in Deliverables D3 and D4, respectively. Instead, it provides a high-level architectural framework that integrates these aspects cohesively.

1.4 Document Structure

This Service Specification document is divided into the following Chapters:

- Chapter 1 - Introduction (current): Provides an overview of the project, detailing the context, objectives, and framework for the implementation and operation of the CopernicusLAC Centre Platform.
- Chapter 2 - System Overview: Offers a comprehensive description of the system objectives, components, and high-level architecture, including key requirements and assumptions.
- Chapter 3 - User Roles: Describes the various user roles within the CopernicusLAC platform, outlining their responsibilities and interactions with the system.
- Chapter 4 - System Components: Delivers detailed descriptions of each system component, including data ingestion, processing, discovery, and user interaction mechanisms.
- Chapter 5 - Data Ingestion: Discusses the technologies and workflows used for data ingestion, including validation, metadata enrichment, and storage.
- Chapter 6 - Data Discovery: Explains the mechanisms for data indexing, search, access, and visualisation, along with integration with other platform components.

- Chapter 7 - Application Packaging: Outlines the process of packaging EO applications, covering technologies like CWL, containers, deployment, and version control.
- Chapter 8 - Technologies and Frameworks: Provides an overview of the key technologies and frameworks that form the foundation of the platform's architecture.
- Chapter 9 - Security and Compliance: Details the security measures and compliance protocols, including data protection, access control, and incident response.
- Chapter 10 - Scalability and Performance: Describes the strategies for ensuring the platform can scale effectively while maintaining optimal performance, including load testing and resource management.

1.5 Applicable Documents

Ref.	Title and References	Issue
[SOW]	CopernicusLAC Infrastructure Support (EOP-EOP-SI-SOW-2023-15)	1.0
[CTR]	ESA Contract No. 4000140709/23/I-EF with Terradue for Platform Extensions with AI Capabilities	1.0
[D1]	T2-ESA-COPLAC-RS-24-028 D1 CopernicusLAC Centre Specification and Performance Requirements	1.4

1.6 Reference Documents

Ref.	Title and References	Issue
[PROP]	T2-ESA-AO1-11947-OF-23-062 Terradue Offer for CopLAC Infra (all Volumes)	1.0
[D3]	T2-ESA-COPLAC-ID-24-030 D3 Interface Control Document	1.3

1.7 Acronyms and Abbreviations

The following acronyms are used within this document:

API	Application Programming Interface
CDSE	Copernicus Data Space Ecosystem
COG	Cloud Optimised GeoTIFF
CWL	Common Workflow Language
DBaaS	Database as a Service

DPO	Data Protection Officer
DRR	Disaster Risk Reduction
EO	Earth Observation
ESA	European Space Agency
EU	European Union
ETL	Extract, Transform, Load
GDPR	General Data Protection Regulation
GUI	Graphical User Interface
HPC	High-Performance Computing
HTTPS	HyperText Transfer Protocol Secure
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
KaaS	Kubernetes as a Service
KPI	Key Performance Indicator
LAC	Latin America and the Caribbean
MTTR	Mean Time to Repair
OGC	Open Geospatial Consortium
PaaS	Platform-as-a-Service
PDP	Personal Data Protection
RBAC	Role-Based Access Control
SFTP	Secure File Transfer Protocol
SLA	Service Level Agreement
STAC	SpatioTemporal Asset Catalog
TLS	Transport Layer Security
WMTS	Web Map Tile Service

2 System Overview

This section provides a comprehensive overview of the Platform system, detailing the objectives, components, high-level architecture, and key requirements. The purpose is to establish a clear understanding of the Platform system's design and its intended functionalities, setting the stage for detailed descriptions in the subsequent sections.

2.1 System Objectives

The primary objectives of the Platform are:

- **Data Accessibility:** Facilitate easy access to Copernicus data for users in the LAC region.
- **Data Processing:** Provide data processing capabilities to transform raw satellite data into usable information for various applications such as disaster risk reduction and recovery.
- **Federation:** Ensure data exchange and interoperability with the Copernicus Data Centre in Chile and other relevant entities.
- **Scalability and Flexibility:** Design a system that is scalable and flexible to accommodate future growth and additional functionalities.
- **User Support and Training:** Support local institutions by providing training and tools necessary for the independent operation of the CopernicusLAC Centre.

The CopernicusLAC Centre implements the tools and activities for:

- **Data hosting:** Store and provide access to Copernicus Sentinel data and services to the LAC region.
- **Data processing:** Development of the middleware needed to process Sentinel and other relevant data.
- **Data exchange:** Federation with the Copernicus Data Centre in Chile, and with other relevant entities.

2.2 User Roles

As defined in D1, the CopernicusLAC platform supports a variety of user roles, each with distinct responsibilities and interactions with the system. These roles ensure the effective operation and utilisation of the platform, facilitating access to, processing, and analysis of EO data. Below is a summary of the primary user roles within the CopernicusLAC platform, highlighting their key activities and how they contribute to the overall functionality and goals of the system:

- **Data Explorer**
 - Uses the CopernicusLAC Portal for data search and retrieval.
 - Utilises catalogue and geospatial tools for data exploration.
 - Performs simple analytics with visualisation and filtering services.

- Conducts analysis within the platform to leverage computational capabilities.
- Service Stakeholder
 - Interacts with service web applications co-designed with stakeholders.
 - Focuses on risk and recovery mapping, vulnerability assessment, and EO monitoring of extreme events.
 - Provides feedback on web application functionality and usability.
 - Applies insights from applications to domain-specific tasks.
- Data Analyst
 - Accesses and manipulates data sets through the CopernicusLAC Portal and APIs.
 - Uses analytical techniques and tools for data processing and analysis.
 - Creates reports and visualisations to present findings.
 - Utilises interactive graphical applications (e.g., QGIS, Panoply) and notebooks for detailed analysis.
- Service Developer
 - Develops algorithms to process and analyse EO data delivered in dedicated web applications in the activities of a parallel project to provide essential EO-based information to specific stakeholders.
 - Tests and refines code using the interactive development environment.
 - Collaborates with Operators to integrate algorithms into workflows.
- Operator
 - Manages deployment and orchestration of containerized applications.
 - Monitors system performance and ensures high availability.
 - Performs regular maintenance and system updates.
 - Deploys new services developed by Algorithm Developers.
 - Addresses technical issues and supports other user roles.

2.3 System Components

The Platform relies on a comprehensive suite of software tools and protocols to streamline various processes involved in EO data management and application development with the objective of providing a Cloud-agnostic EO Data Hosting and Exploitation environment. The components are delivered as a fully-fledged Platform-as-a-Service (PaaS) that supports:

- **Efficiency in Data Handling:** Streamline the process of EO data ingestion and calibration, ensuring data integrity and accuracy.
- **Discoverability:** Facilitate swift and precise data searches, aided by a geospatial and temporal querying mechanism.
- **Flexibility in Integration:** Allow integration of external tools and data processing applications.
- **Versatile Execution:** Provide a diverse execution environment ranging from local servers to cloud-based infrastructures.
- **User Engagement:** Foster an interactive and intuitive user experience, offering a platform where applications can be easily searched, reviewed, and executed.

The diagram below shows the several key components, each fulfilling a specific role in the overall architecture.

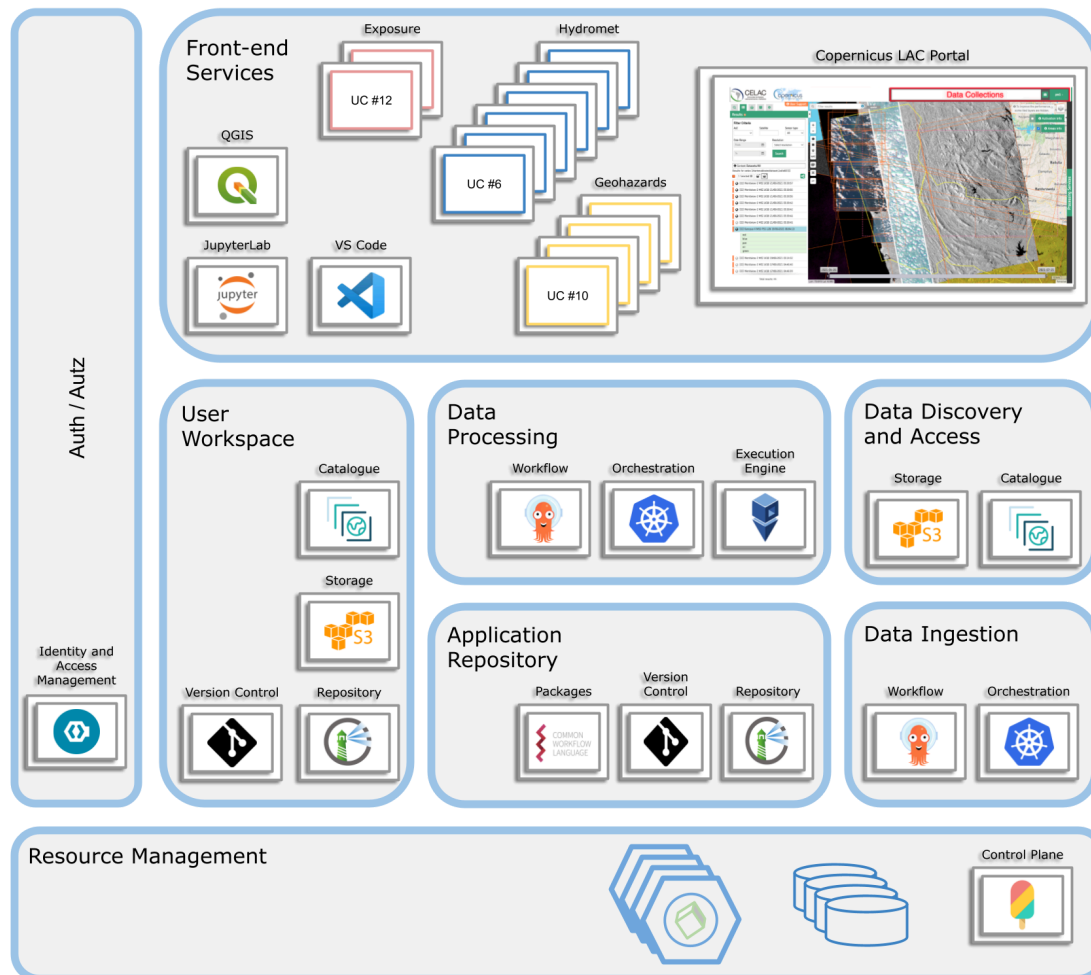


Figure 1 - CopernicusLAC Centre Platform System Components

Key components:

- **Data Ingestion:** Manages the ingestion of data from external sources, ensuring that data is validated and pre-processed before being stored or used.
- **Data Discovery and Access:** Stores metadata about available data, manages the actual data storage, and provides tools for users to search, discover, and visualise data. It also manages secure access to data.
- **Data Processing:** Facilitates communication between different processing components, manages workflows for processing data, and provides access to various data processing services, supporting additional functionalities as needed.
- **Application Repository:** Stores packages of applications used for processing and analysis, manages containers for running applications, and ensures version control for applications and workflows.

- **Front-end Services:** Acts as the main interface for users to interact with the platform, including specific web applications designed for different use cases, tools for interactive data analysis (such as QGIS, JupyterLab, and VS Code), a development environment for creating and testing applications, and support for interactive computational notebooks for data analysis.
- **User Workspace:** Provides users with personal workspaces that include version control, storage, and repositories for managing their work and data.
- **Authentication and Authorization:** Manages user authentication and authorization, ensuring secure access to the platform.
- **Resource Management:** Oversees the allocation and management of system resources, ensuring operation and scalability.

2.4 High-Level Architecture Diagram

The high-level functional diagram shown in Figure 2 provides a visual representation of the system's structure, highlighting the main components and their interactions. It serves as a blueprint for understanding how data flows through the system and how the different modules interact with each other.

The architecture of the platform is designed with an array of interfaces and integrations, ensuring operations throughout the data acquisition, storage, and retrieval processes. One of the core features of this architecture is its diverse set of API endpoints. These endpoints play a pivotal role in handling real-time data streaming from various platforms and stakeholders.

The selected design leverages cloud-agnostic resources with modern technologies and standards to optimise data discovery, access, application development, hosting, and execution that are being used in several operational platforms.

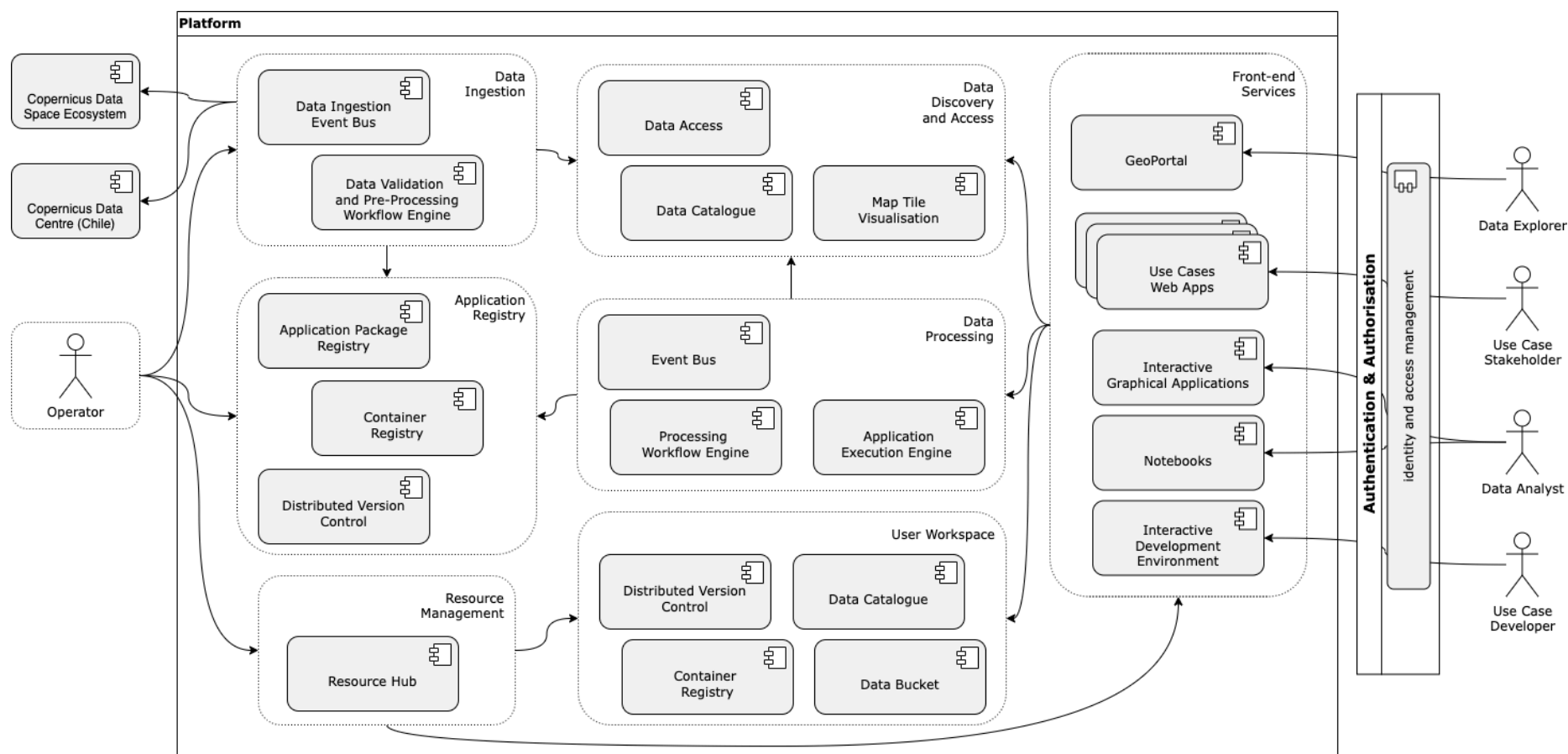


Figure 2 - CopernicusLAC Center Platform Functional Diagram and main use cases

2.5 Assumptions and Constraints

One of the key assumptions underlying the design and deployment of the Platform in the CopernicusLAC Centre is that the system will be deployed in an infrastructure that provides Kubernetes as a Service (KaaS) and Databases as a Service (DBaaS). This assumption has several implications for the system design, scalability, maintenance, and overall operational efficiency:

- **Kubernetes as a Service (KaaS):** The use of KaaS implies that the management and orchestration of containerized applications are handled through a cloud service provider. This facilitates ease of deployment, scalability, and management of the system's applications and services. Kubernetes provides a framework for automating deployment, scaling, and operations of application containers across clusters of hosts.
- **Object storage as a Service:** The use of object storage enables data management through a highly available S3 compatible API, allowing to benefit from a performant and scalable access to large amounts of unstructured data, from anywhere in the platform. This choice ensures that data retrieval is both swift, reliable and, in keeping with modern geospatial data practices, supports Cloud-Optimised Geotiffs (CoG). In addition to the primary object storage layer, the infrastructure expects also to interoperate with a deep/cold storage archive tier, such as those offered by commercial cloud providers. Data recovered from cold storage will be staged back into the primary S3-compatible object storage layer for access and processing. The associated access latencies and management workflows will be handled at the orchestration level.
- **Databases as a Service (DBaaS):** The assumption that databases are provided as a service indicates that the data storage and management aspects of the system are hosted and managed by a dedicated provider. This service typically includes database setup, maintenance, backups, and scalability, thereby reducing the overhead on our system for these tasks. DBaaS ensures high availability, performance, and scalability of the database integral to the system. The preferred DBaaS technology is PostgreSQL, which is assumed to be available as a managed relational database service from the hosting provider. This choice is aligned with the implementation details provided in the component-specific architecture sections (3.2.4, 3.4.4, 3.6.4) and the technology stack outlined in Section 7.

Given these infrastructure assumptions, the system is designed with certain constraints and considerations:

- **Dependency on Resource Provider:** Relying on Kubernetes as a service and databases as a service means the system's performance and availability are partly contingent on the reliability and capabilities of the provider.

- Scalability and Resource Management: While these services provide scalability, there is a dependency on their ability to allocate resources. Any limitations in their scalability could impact the performance of the Platform services.

2.6 Summary of Key Requirements

As defined in D1, the system must meet several requirements to ensure it fulfils its objectives:

2.6.1 Data Hosting

- Ability to ingest, host and manage large volumes of Copernicus data, including Sentinel-1, Sentinel-2, Sentinel-3, and Sentinel-5P archives.
- Mechanisms for data synchronisation and timely updates.

2.6.2 Data Exploration and Processing

- A data catalogue with search, access, and retrieval functionalities.
- Scalable processing environment supporting both on-demand and systematic processing workflows.
- Integration of standard EO processing toolboxes and user-friendly data ingestion mechanisms.

2.6.3 Federation

- Data exchange protocols and interoperability with the Copernicus Data Centre in Chile.
- Capability to federate services and data with other relevant entities.

2.6.4 Security and Compliance

- Adherence to data protection regulations and security frameworks.
- Implementation of measures to ensure data confidentiality, integrity, and availability.

2.6.5 Scalability and Performance

- The system must support up to 3000 users with the ability to scale up as the user base grows.
- High performance in data processing and retrieval to meet user demands.

2.6.6 User Support

- Provision of comprehensive user manuals, training materials, and technical support to ensure effective use of the system.

- Development of a training and transfer plan to facilitate the handover of operations to local institutions.

3 System Components

This section provides a detailed description of each system component within the CopernicusLAC platform. For each component, we outline its purpose, the implementation strategy, integration points with other components, and the software stack utilised. This approach ensures a comprehensive understanding of the individual parts and their contributions to the overall system architecture.

Each component plays a role in achieving the system's objectives, from data ingestion and processing to storage and user interaction. The detailed breakdown includes:

- **Purpose:** Explains the primary function and goals of the component within the system.
- **Implementation:** Describes how the component is realised, including the technologies and methodologies employed.
- **Integration Points:** Identifies how the component interacts with other parts of the system, ensuring data flow and functionality.
- **Software Stack:** Lists the specific software tools, frameworks, and platforms used in the implementation of the component.

By breaking down the system into these detailed components, we aim to provide a clear and thorough understanding of the CopernicusLAC platform's architecture, facilitating effective development, deployment, and maintenance.

3.1 Data Ingestion

3.1.1 Purpose

The Data Ingestion component is crucial for the CopernicusLAC platform as it manages the initial intake of data from external sources. Its primary purpose is to support a flexible data ingestion strategy that accommodates multiple levels of ingestion, tailored to the dataset's role, user requirements, and operational scenarios. These levels include:

1. **Metadata Registration:** Ensuring that datasets hosted externally are discoverable by registering their metadata and access points without duplicating the original data.
2. **Metadata Registration with Dataset Copying:** Facilitating local storage of datasets when frequent access or integration with platform workflows is required.
3. **Metadata Registration with Dataset Calibration:** Supporting pre-processing or transformation of datasets during ingestion to align with specific analytical or quality requirements.

This component ensures that incoming data, whether stored locally or accessed externally, is validated, indexed, and made accessible for subsequent processing and analysis.

3.1.2 Implementation

The implementation of the Data Ingestion component combines automated pipelines and workflow management systems to ensure the systematic and reliable intake of data from external sources, such as the Copernicus Data Space Ecosystem and the Copernicus Data Centre in Chile. The component supports multiple levels of ingestion, allowing the platform to adapt dynamically to various operational needs and dataset requirements:

- **Automated Data Ingestion Pipelines:** These pipelines are designed to handle large volumes of incoming data automating the processes of metadata harvesting, dataset copying, and, when required, dataset calibration. The ingestion workflows are flexible, dynamically selecting the appropriate level of ingestion based on the dataset's purpose:
 - For metadata registration, pipelines validate and catalog metadata for discovery without copying the original data.
 - For metadata registration with dataset copying, pipelines download and validate the datasets before storing them locally.
 - For calibration scenarios, additional processing steps, such as format conversion or georeferencing, are applied before registration and storage.
- **Data Validation and Pre-Processing:** Validation checks are integral to all ingestion levels, ensuring the integrity, completeness, and quality of ingested data. For metadata-only ingestion, the system focuses on verifying compliance with the STAC specification. For datasets requiring copying or calibration, validation includes checksum verification and content-specific checks to confirm readiness for storage and analysis.
- **Event-Driven Architecture:** An event-driven approach underpins the ingestion system, utilizing event buses to manage the flow of data through the pipeline. This architecture supports scalable and flexible data handling, dynamically adjusting to variations in data loads, such as high-frequency acquisitions or large-volume transfers.
- **Standards Compliance:** The pipelines workloads are developed using CWL and adhere to the OGC Best Practices for EO Application Package to ensure portability and scalability.

3.1.3 Integration Points

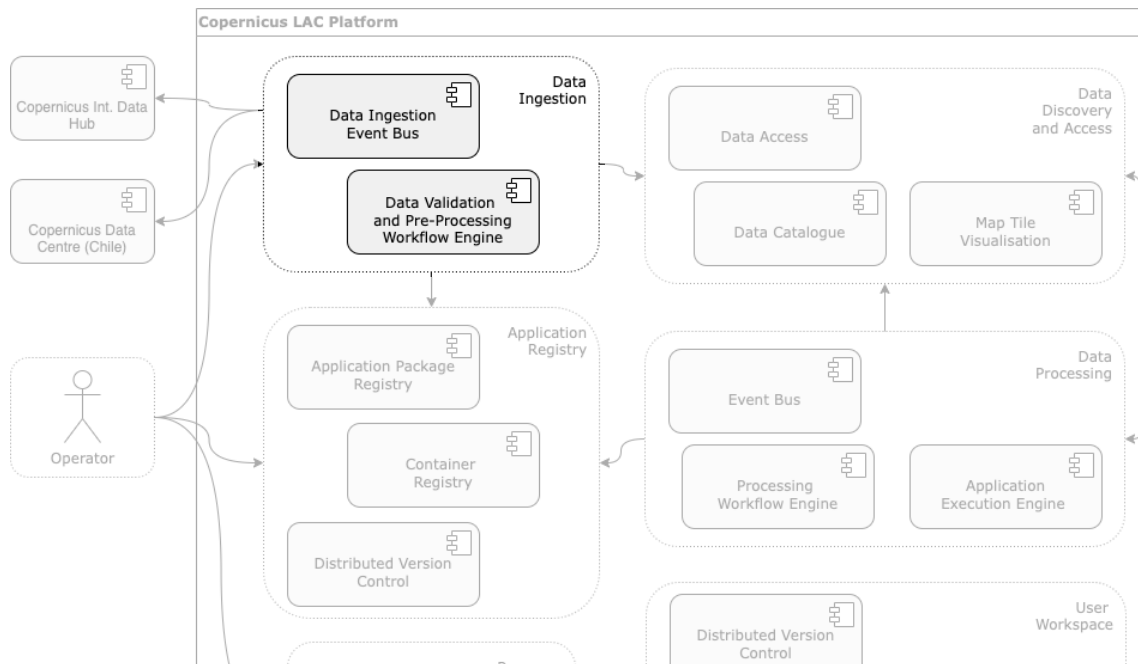


Figure 3 - Data Ingestion integration points

The Data Ingestion component integrates with three other components within the CopernicusLAC platform to ensure data flow and interoperability:

- **Copernicus Data Space Ecosystem and Copernicus Data Centre (Chile):** These external sources provide the raw satellite data that is ingested into the platform.
- **Application Repository:** The Data Validation and Pre-processing Workflow Engine retrieves ETL (Extract, Transform, Load) applications from the Application Repository to manage the ingestion for all specific data collections or product types.
- **Data Discovery and Access:** Once data is ingested and validated, it is stored and registered within the Data Discovery and Access component, making the data and web map tiles available and catalogued for user access.

3.1.4 Software Stack

The Data Ingestion component utilises a range of software tools and technologies to achieve its objectives:

- **Event Bus:** Apache Kafka is used for managing the Argo Events event-driven data ingestion process bus. Kafka's capabilities in handling real-time data streams make it an ideal choice for this component.
- **Workflow Orchestration:** Argo Workflows is employed to orchestrate the data ingestion workflows. Argo Workflows' Kubernetes-native design provides scalability and integration with containerized applications.

- **Data Validation and Pre-Processing:** Custom Python scripts and ETL tools are used to validate and pre-process the incoming data. These tools ensure that the data meets the required quality standards before being stored or further processed.
- **Common Workflow Language (CWL):** CWL is used to define and execute the data ingestion pipelines, ensuring portability and scalability.
- **OGC Best Practices for EO Application Package:** The ingestion pipelines adhere to these best practices, ensuring compatibility and standardisation across the platform.
- **Container Orchestration:** Kubernetes is used for deploying and managing the containerized ingestion pipelines. Kubernetes provides the necessary scalability and reliability to handle large-scale data ingestion tasks.
- **Storage Solutions:** The platform employs S3-compatible object storage for data preservation, utilising its features like data versioning, lifecycle policies, and security mechanisms to ensure secure data management.
- **SpatioTemporal Asset Catalog (STAC):** STAC specifications are used to organise and describe the data, making it easily searchable and accessible. Each dataset is accompanied by a corresponding STAC item, ensuring standardised metadata representation.

3.2 Data Discovery and Access

3.2.1 Purpose

The Data Discovery and Access component's primary purpose is to enable users to search, discover, and access the EO data ingested into the system. This component ensures that the data is organised, searchable, and retrievable in a user-friendly manner, supporting the platform's goals of providing accessible and usable data for various applications.

3.2.2 Implementation

The implementation of the Data Discovery and Access component involves several key elements designed to facilitate effective data search and retrieval:

- **Indexing and Search:** PostgreSQL, provided as a service by the cloud provider, is used to index and store the metadata catalogued during the ingestion process. PostgreSQL's querying capabilities enable fast search operations. Users can perform queries based on various criteria such as time, location, and data type.
- **API Access:** The platform provides APIs that allow users to programmatically search and retrieve data. These APIs adhere to industry standards, ensuring broad compatibility and ease of use. The STAC API implementation with FastAPI, using PgSTAC for backend storage, ensures data querying and retrieval.
- **Visualisation Tools:** Integrated visualisation tools enable users to preview and interact with the data directly within the platform. This includes map-based interfaces and other graphical tools that help users understand and analyse the data. The Web

Map Tile Service (WMTS) API, powered by a Titiler backend, ensures accurate and responsive data visualisation.

- **Signed URL Service:** For data stored in S3-compatible object storage, the platform uses a signed URL service to securely provide access to the data. This service generates temporary URLs that allow users to download data directly from storage.

3.2.3 Integration Points

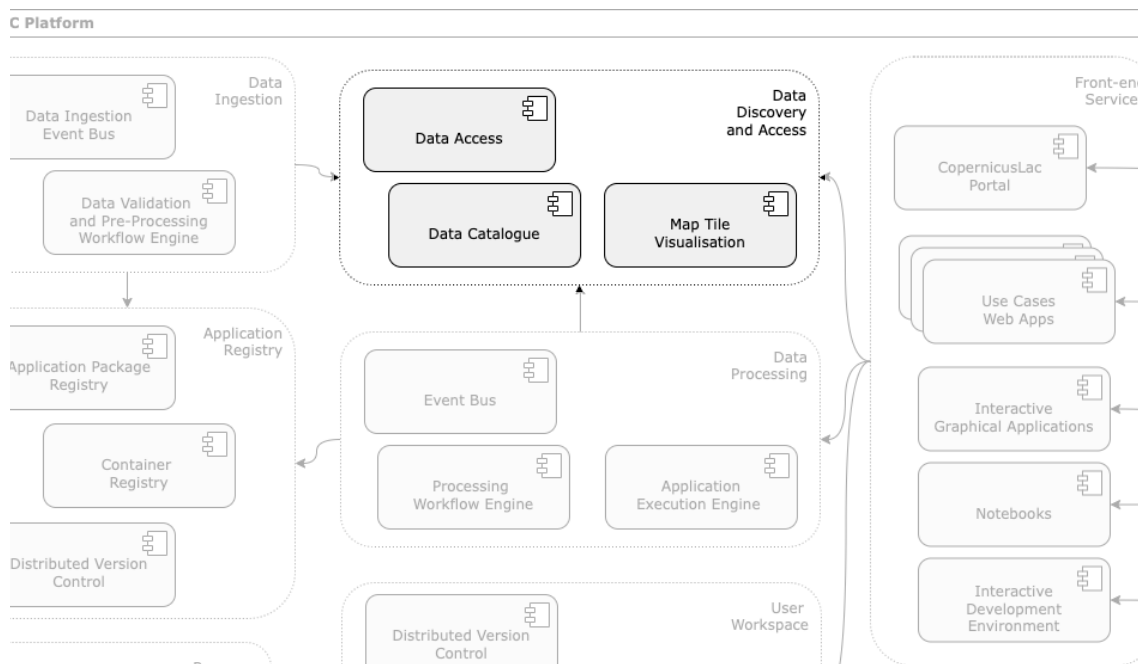


Figure 4 - Data Discovery and Access integration points

The Data Discovery and Access component integrates with multiple other components within the CopernicusLAC platform to ensure data flow and user interaction:

- **Data Ingestion Component:** Provides metadata from the ingestion process to be indexed for discovery.
- **Data Processing Component:** Facilitates the discovery of processed data products, making them available for user access and further analysis.
- **Front-end Services:** Provides interfaces for users to search, visualise, and access data. This includes integration with the CopernicusLAC Portal and other web applications.
- **User Workspace:** Allows users to save search results and access data within their personal workspaces for further analysis and processing.
- **Data Visualization Component:** Uses visualisation links of each data item referenced in the Data Catalog using the Web Map Links and Rendering STAC extensions.

- **Data Download Functionality:** Supports any data download functionality from any of the front-end services or data processing components, ensuring services and users can access the data they need.

3.2.4 Software Stack

The Data Discovery and Access component utilises a range of software tools and technologies to achieve its objectives:

- **PostgreSQL:** Utilised for indexing and storing metadata. PostgreSQL's advanced querying capabilities enable search and retrieval of metadata.
- **PostGIS:** used to extend the capabilities of the PostgreSQL relational database by adding support for storing, indexing, and querying geospatial data
- **PgSTAC:** Stores all collection and item records as JSONB, allowing for any STAC custom fields to be stored and retrieved transparently.
- **STAC API with FastAPI:** Provides a standardised interface for programmatic access to the catalogued data, enabling users to perform detailed searches and retrieve metadata and data assets. The FastAPI interface validates requests and data sent to the PgSTAC backend.
- **Data Storage Solutions:** S3-compatible object storage is used to store and manage the actual data files, ensuring scalable data access.
- **Tileiter:** Backend service for serving map tiles based on OGC Web Map Tile Service (WMTS) requests, supporting dynamic generation of tiles from COG products.
- **Signed URL Service:** Generates temporary, secure URLs that allow users to download data directly from the S3 storage, ensuring secure and controlled access.

3.3 Data Processing

3.3.1 Purpose

The Data Processing component of the CopernicusLAC platform is designed to transform raw EO data into valuable information and insights. This component enables users to perform various data processing tasks, including data cleaning, transformation, analysis, and visualisation. The primary purpose of this component is to provide a scalable infrastructure that supports both systematic and on-demand processing of EO data, ensuring that users can derive meaningful insights and actionable information from the data.

3.3.2 Implementation

The implementation of the Data Processing component involves several key elements designed to facilitate scalable data processing:

- **Event Bus:** The Argo Events event bus, powered by Apache Kafka, handles the communication between different parts of the system. It ensures that data is passed

from the ingestion component to the processing workflows, and it also handles notifications and triggers for processing tasks.

- **Processing Workflow Engine:** The processing workflow engine, implemented using Argo Workflows, manages the execution of complex processing pipelines. These pipelines are defined using the CWL and orchestrate various tasks such as data validation, transformation, and analysis.
- **Application Execution Engine:** The application execution engine provides the web service interface (OGC API Processes) that is called by the front-end services to execute specific data processing applications. It also calls the Processing Workflow Engine to execute EO applications defined as application packages. The application execution engine has two distinct functionalities: deployment and execution.

The event bus is responsible for handling the communication and data flow between different components of the data processing pipeline. Argo Events uses Apache Kafka to manage the flow of messages and events, ensuring that data is routed and processed in a timely manner.

- **Data Flow Management:** The event bus manages the flow of data from the Data Discovery and Access Component to the processing workflows and applications.
- **Notification and Triggering:** It handles notifications and triggers for processing tasks, ensuring that workflows are executed when data becomes available.

The processing workflow engine manages the execution of complex data processing workflows. It uses Argo Workflows to define and orchestrate these workflows, ensuring that tasks are executed in the correct order and that dependencies are managed.

- **Workflow Definition:** Workflows are defined using the CWL, providing a standard and portable way to specify processing tasks.
- **Orchestration:** Argo Workflows orchestrates the execution of these workflows, managing dependencies, scheduling tasks, and handling retries and failures.
- **Event-Driven Workflow:** Argo Events ensures that specific processes or tasks are initiated based on predefined events, maintaining system responsiveness and efficiency.

The Application Execution Engine provides the web service interface (OGC API Processes) that is called by the front-end services to execute specific data processing applications. It manages the execution of these applications, ensuring they are accessible via standardised APIs, and it calls the Processing Workflow Engine for executing complex workflows.

- **Deployment:**
 - **Functionality:** The deployment functionality handles the submission of EO application packages described using CWL. This involves specifying parameters, software items, executables, dependencies, and metadata.

- **Interface:** The deployment interface is defined in the OGC API Processes Part 2 standard, outlining the protocols and methods for deploying EO applications within the environment.
- **Setup:** Ensures applications are set up in the environment and ready for execution.
- Execution:
 - **Functionality:** The execution functionality initiates and manages the processing jobs. Users interact with this functionality to execute the deployed EO applications, providing necessary input parameters and initiating the processing tasks.
 - **Interface:** The execution interface is aligned with the guidelines set in OGC API Processes Part 1, detailing how users can initiate and manage processing jobs within the environment.
 - **Management:** Handles task scheduling, monitoring, and the retrieval of outputs.

3.3.3 Integration Points

The Data Processing component integrates with multiple other components within the CopernicusLAC platform to ensure data flow and interoperability:

- Data Discovery and Access Component: Receives data products from the Data Discovery and Access Component for processing. Processed data products are indexed and catalogued for easy search and retrieval.
- User Workspace: Provides a space where users can manage and execute their data processing tasks, as well as access and visualise the results of their processing jobs.
- Front-end Services: Interfaces with the CopernicusLAC Portal and other web applications to allow users to submit processing jobs, monitor their progress, and access the results through the OGC API Processes interface.
- Data Visualization Component: Supports the visualisation of processed data, enabling users to interact with and analyse the results.
- Application Repository: Integrates with the Application Repository to access the application code and containers necessary for executing the EO applications.

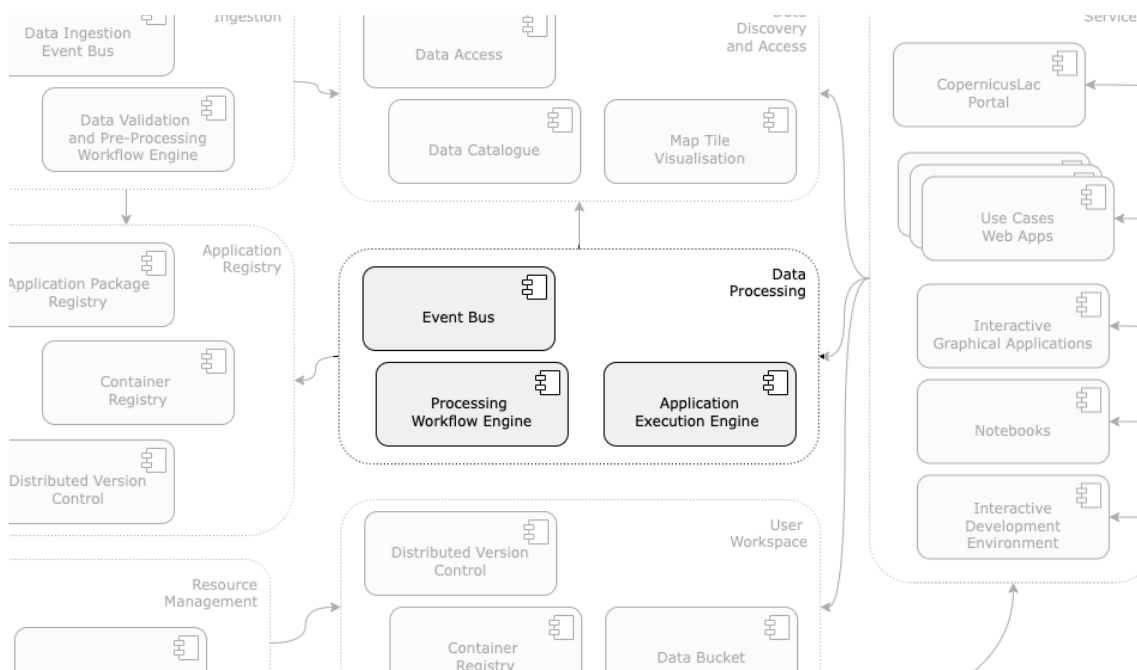


Figure 5 - Data Processing integration points

3.3.4 Software Stack

The Data Processing component utilises a range of software tools and technologies to achieve its objectives:

- Calrissian¹: a CWL runner for kubernetes used to submit and manage data processing workflow executions on Kubernetes, ensuring portability and scalability.
- Argo Workflows: Orchestrates the execution of CWL-defined processing pipelines, providing scalable workflow management.
- Argo Events: Provides an event-driven framework to trigger workflows based on specific events, ensuring responsive and automated processing.
- Docker, podman: Containerizes processing tasks to ensure consistent and reproducible execution across different environments.
- Kubernetes: Manages and orchestrates the execution of containerized processing tasks, providing dynamic scaling and reliability.
- Python and GDAL: Commonly used for developing custom data processing scripts and tools, leveraging the extensive libraries and capabilities of Python and the Geospatial Data Abstraction Library (GDAL).
- S3-Compatible Object Storage: Stores processed data, ensuring scalable data access and management.
- Zoo Project is a software solution that implements the OGC API Processes. This standard provides a standardised web service interface for initiating and managing data processing tasks from front-end services.

¹ <https://github.com/Duke-GCB/calrissian>

- Deployment Interface (Part 2): Defines the standards and protocols for deploying EO applications within the environment.
- Execution Interface (Part 1): Specifies the standards for executing the deployed applications, detailing how users can initiate and manage processing jobs.

3.4 Application Registry

3.4.1 Purpose

The Application Registry component of the CopernicusLAC platform is designed to manage the storage, organisation, and retrieval of EO application packages. This component ensures that application packages, including their metadata, code, configuration files, and container images, are securely stored and easily accessible for deployment and execution within the platform. The primary purpose of the Application Registry is to provide a centralised repository that supports the deployment and execution of EO applications, promoting consistency, reproducibility, and efficiency in data processing workflows.

3.4.2 Implementation

The implementation of the Application Registry involves several key elements designed to facilitate the management and retrieval of EO application packages:

- GitLab for Application Packages: GitLab is used to store the application packages as Continuous Integration pipeline artifacts, including the CWL descriptions, code, and configuration files. This ensures that all components required for the execution of EO applications are version-controlled and easily accessible.
- Harbor as a container registry: Harbor serves as the cloud-native container registry to store and manage the container images associated with the application packages. This ensures that the application containers are easily accessible and deployable.
- Metadata Management: Comprehensive metadata for each EO application package is stored within GitLab, including details such as version, dependencies, configuration parameters, and usage instructions.
- Version Control: GitLab provides version control for application packages, allowing users to track changes, revert to previous versions, and ensure compatibility with different processing environments.
- Access Control: Both GitLab and Harbor implement access control mechanisms to ensure that only authorised users can add, update, or retrieve application packages and container images. This ensures the security and integrity of the stored applications.
- Search and Retrieval: GitLab's integrated search functionality allows users to quickly locate and access the application packages they need for deployment and execution. This may evolve towards a dedicated document oriented database.

3.4.3 Integration Points

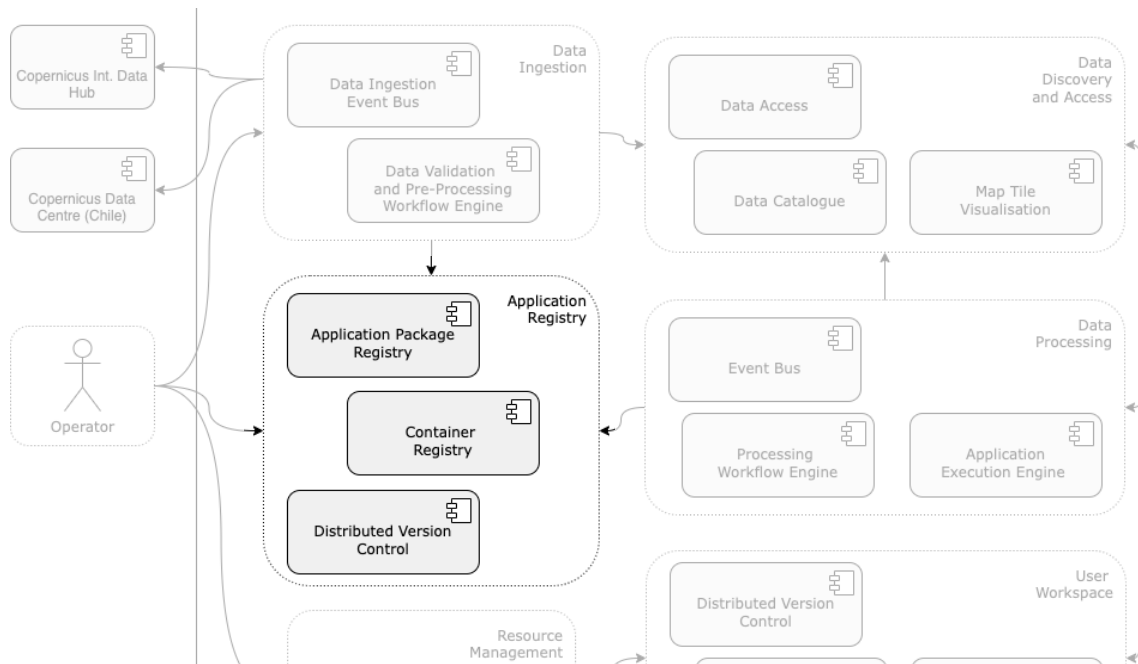


Figure 6 - Application Registry integration points

The Application Registry integrates with multiple other components within the CopernicusLAC platform to ensure application management and execution:

- **Application Execution Engine:** Retrieves application packages from GitLab and deploys these as OGC API Processes. The registry ensures that the necessary application code, containers, and configurations are available to execute EO applications.
- **Processing Workflow Engine:** The Processing Workflow Engine accesses the application packages and container images in the registry to orchestrate complex processing workflows, ensuring that all required dependencies and configurations are met.
- **User Workspace:** Users interact with the Application Registry to retrieve operational application packages and copy them to their own private space. Each user workspace has dedicated software repositories in GitLab and a dedicated Harbor based container registry to manage private and operational packages and associated container registries.
- **Data Processing Component:** The interaction with the Application Registry for the deployment and execution of applications is handled indirectly through the Data Processing Component, which orchestrates the retrieval and deployment of EO application packages on the Application Execution Engine.
- **Data Ingestion Component:** Similar to the Data Processing component, the Data Ingestion component interacts with the Application Registry to retrieve application packages required for the ingestion, validation, and pre-processing of EO data.

3.4.4 Software Stack

The Application Registry utilises a range of software tools and technologies to achieve its objectives:

- GitLab: Used to store the application packages, including CWL descriptions, code, and configuration files. It provides version control and metadata management.
- Harbor: Used to store and manage container images associated with application packages, ensuring that containers are easily accessible and deployable.
- Metadata Database and Search: PostgreSQL, provided as a service by the cloud provider, is used within GitLab to manage the metadata associated with each application package and to enable querying capabilities.
- Access Control: Implemented using IAM (Identity and Access Management) solutions within GitLab and Harbor to ensure secure access to the registry.

3.5 Front-end Services

3.5.1 Purpose

The Front-end Services component of the CopernicusLAC platform is designed to provide users with intuitive and accessible interfaces to interact with the platform's capabilities. These services include the CopernicusLAC Portal, interactive graphical applications (e.g., QGIS, Panoply), notebooks (e.g., Jupyter), and an interactive development environment (e.g., VSCode). The primary purpose of the Front-end Services is to enable users to utilise the platform's resources and capabilities through user-friendly interfaces.

3.5.2 Implementation

The implementation of the Front-end Services involves several key elements designed to ensure an efficient user experience:

- CopernicusLAC Portal: A centralised web-based portal that provides users with access to data discovery, retrieval, and visualisation tools. The portal offers an intuitive user interface with clear categorizations, filters, and search capabilities.
- Interactive Graphical Applications: Tools like QGIS and Panoply are integrated to provide users with advanced capabilities for visualising and analysing geospatial data.
- Notebooks: Platforms like Jupyter Notebooks allow users to perform complex data analyses and visualisations through an interactive, code-based interface. These notebooks can access data directly from the User Workspace.
- Interactive Development Environment (IDE): Tools like VSCode are provided to enable users to develop, test, and debug their applications in a user-friendly environment.
- User Authentication and Authorization: Secure access to front-end services through integrated authentication and authorization mechanisms.

- **Visualisation Clients:** Integrated visualisation clients access the Titiler backend in the Data Discovery and Access component to convert geospatial raster data into web-friendly map tiles for display on web applications.
- **Use Case / Service Web Applications:** These applications are being developed in parallel projects to provide essential EO-based information co-designed with stakeholders. They enhance the platform's relevance and usability in addressing key areas such as disaster risk reduction and climate event monitoring.
- **Resource Management:** The resource management component instantiates or deploys the front-end services, ensuring that they are properly configured and available for user access.

3.5.3 Integration Points

The Front-end Services integrate with multiple other components within the CopernicusLAC platform to provide a cohesive user experience:

- **User Workspace:** The front-end services interact with the User Workspace to access data, execute processing tasks, and retrieve results. This integration allows users to manage their data and tasks through the front-end interfaces.
- **Data Discovery and Access Component:** Enables users to search and retrieve EO data. The front-end services provide interfaces for users to explore and visualise this data. Visualisation clients within the front-end services access the Titiler backend here for dynamic map tile generation.
- **Data Processing Component:** Allows users to submit processing jobs and monitor their progress. The front-end services provide interfaces for defining processing tasks and viewing results.
- **Application Registry:** Facilitates the deployment and execution of application packages. The front-end services provide interfaces for users to select and manage applications from the registry.
- **Security Services:** Integrates with authentication and authorization services to ensure secure access to the platform's resources.
- **Resource Management:** Coordinates with the resource management component to ensure that front-end services are properly instantiated, configured, and maintained.

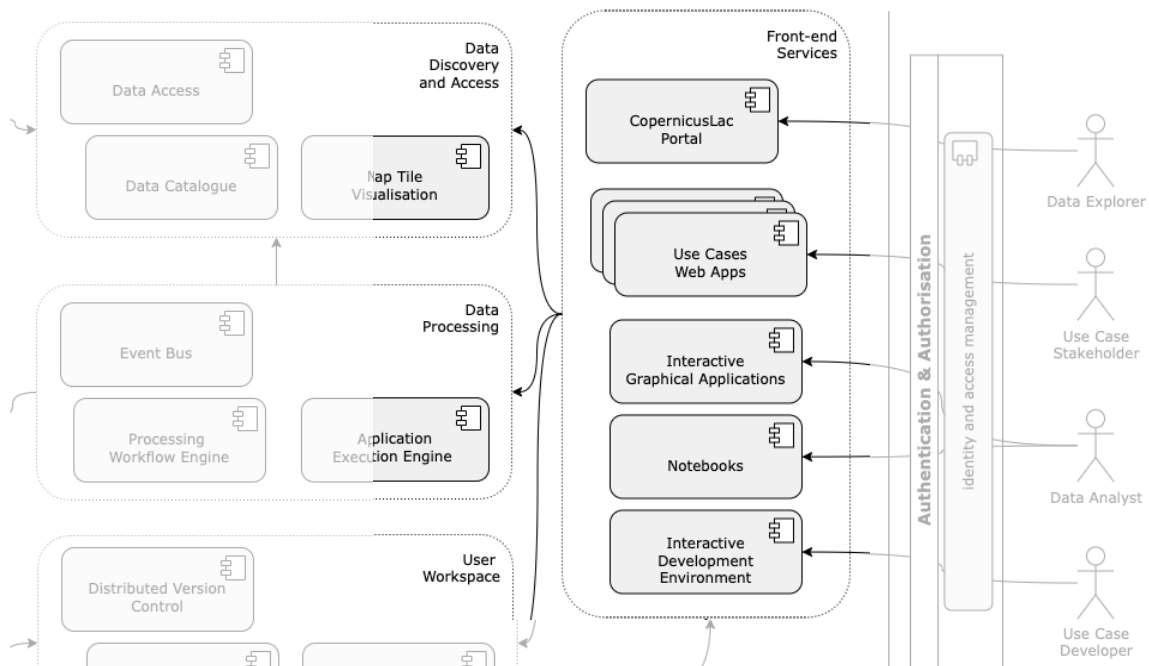


Figure 7 - Front-end Services integration points

3.5.4 Software Stack

The Front-end Services utilise a range of software tools and technologies to achieve their objectives:

- **Web Frameworks:** Tools such as React or Angular are used to develop responsive and interactive web applications.
- **Visualisation Libraries:** Libraries such as D3.js, Leaflet, or OpenLayers are used to create interactive maps and visualisations.
- **JupyterHub:** JupyterHub provides access to the platform's services and web applications in a Software-as-a-Service (SaaS) mode. Through JupyterHub, users can engage with containerized Interactive Graphical Applications (IGAs), specialised geospatial data exploration web apps, and customizable dashboards, enhancing their ability to interact with and analyse EO data.
- **Notebooks:** JupyterHub is used to manage and provide access to Jupyter notebooks in a Software-as-a-Service (SaaS) mode. This setup allows users to perform interactive, code-based data analysis and visualisation within their web browsers, leveraging containerized environments for consistent performance and scalability across multiple users.
- **Interactive Development Environment (IDE):** Tools like VSCode are provided for application development, testing, and debugging.
- **Authentication and Authorization:** Tools such as OAuth or OpenID Connect are used to manage user authentication and authorization.

- APIs: RESTful APIs are used to interact with backend services, facilitating data access, processing job submission, and retrieval of results.
- Containerization: Docker is used to containerize front-end services, ensuring consistent deployment and scalability.
- Continuous Integration/Continuous Deployment (CI/CD): Tools such as GitLab CI/CD are used to automate the deployment and updating of front-end services.
- Visualisation Clients: Tools that interact with the Tiler backend in the Data Discovery and Access component for generating and displaying map tiles.

3.6 User Workspace

3.6.1 Purpose

The User Workspace component of the CopernicusLAC platform is designed to provide users with a personalised environment where they can manage, execute, and analyse their data processing tasks. This workspace offers tools and interfaces that enable users to access EO data, run processing jobs, and manage their own application packages. The primary purpose of the User Workspace is to enhance user productivity by offering a customizable and secure space tailored to individual needs and workflows.

3.6.2 Implementation

The implementation of the User Workspace involves several key elements designed to facilitate a user-friendly working environment:

- Personalised Environment: Each user is provided with a personalised workspace where they can manage their data processing tasks, access data, and analyse results.
- Application Management: Users can manage their own application packages, including adding new applications, updating existing ones, and retrieving applications for processing tasks. Each workspace has its own instance of version control (GitLab) and container registry (Harbor).
- Data Access and Management: The workspace provides CLI clients for users to access and manage EO data stored within the platform. This includes tools for data search, retrieval, and processing.
- Processing Job Management: Users can initiate, monitor, and manage their processing jobs directly from the workspace. This includes submitting jobs, tracking progress, and retrieving outputs.
- Infrastructure Management: Each user has a dedicated namespace on the Kubernetes cluster managed by Crossplane, which provisions necessary resources such as S3 buckets and Helm releases.

- **Synchronisation:** Continuous delivery synchronises the manifests from user repositories with the Kubernetes cluster, ensuring the desired state is consistently applied.
- **Secret Management:** Sensitive information and secrets are managed using the External Secrets Operator (ESO), ensuring secure storage and retrieval.
- **RESTful API:** A RESTful workspace API allows for bootstrapping the user environment, managing manifests, and accessing information about provisioned resources.

3.6.3 Integration Points

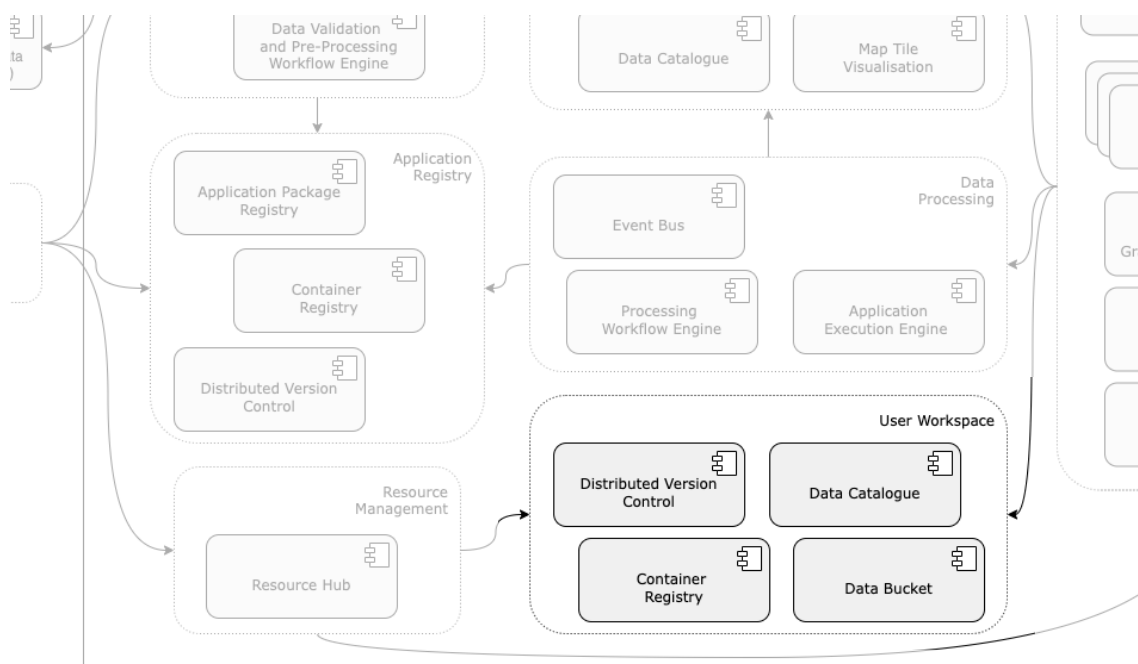


Figure 8 - User Workspace integration points

The User Workspace integrates with multiple other components within the CopernicusLAC platform to provide user experiences for:

- **Data Discovery and Access Component:** Provides users with access to the EO data stored within the platform. Users can search, retrieve, and process data using CLI clients directly from their workspace.
- **Data Processing Component:** Allows users to submit processing jobs and manage workflows. The workspace interfaces with the Data Processing Component to execute user-defined tasks.
- **Application Registry:** Users retrieve application packages from the Application Registry to their workspace. This enables them to manage their private copies of application packages, including version control and container management.

- ArgoCD: Synchronises the user repositories with the Kubernetes cluster to ensure that the desired state of resources and applications is consistently applied.
- Crossplane: Manages the provisioning of infrastructure resources such as S3 buckets, Helm releases, and other cloud resources within the user's namespace.
- Front-end Services: The front-end services, such as notebooks and visualisation tools, utilise the User Workspace to access data and execute processing tasks on behalf of the user.

3.6.4 Software Stack

The User Workspace utilises a range of software tools and technologies to achieve its objectives:

- PostgreSQL: Utilised for indexing and storing metadata. PostgreSQL's advanced querying capabilities enable search and retrieval of metadata.
- PostGIS: used to extend the capabilities of the PostgreSQL relational database by adding support for storing, indexing, and querying geospatial data
- PgSTAC: Stores all collection and item records as JSONB, allowing for any STAC custom fields to be stored and retrieved transparently.
- STAC API with FastAPI: Provides a standardised interface for programmatic access to the catalogued data, enabling users to perform detailed searches and retrieve metadata and data assets. The FastAPI interface validates requests and data sent to the PgSTAC backend.
- GitLab: Used for version control, allowing users to manage their application packages and track changes.
- Harbor: Used to store and manage container images associated with user-defined applications.
- Data Storage: S3-compatible object storage or other scalable storage solutions are used to store user data and processing results.
- CLI Clients: Provides command-line tools for data access, discovery, and processing.
- Processing Job Management: Interfaces with the Data Processing Component to manage the submission, monitoring, and retrieval of processing jobs.
- Kubernetes: Each user has a dedicated namespace on the Kubernetes cluster, managed by Crossplane.
- Crossplane: Manages the provisioning of infrastructure resources and Helm releases, ensuring that users have the necessary resources for their tasks.
- External Secrets Operator (ESO): Manages sensitive information and secrets, ensuring secure storage and retrieval.
- RESTful API: Provides programmatic access to workspace functionalities, allowing for user and admin operations.

3.7 Authentication and Authorization

3.7.1 Purpose

The Authentication and Authorization component of the CopernicusLAC platform is designed to ensure secure access to the platform's resources and services. This component manages user identities, controls access to various system functions, and ensures that only authorised users can access sensitive data and execute specific tasks. The primary purpose of this component is to provide a scalable security framework that protects the platform and its users from unauthorised access and potential security threats.

3.7.2 Implementation

The implementation of the Authentication and Authorization component involves several key elements designed to provide a secure and user-friendly experience, utilising the full stack of services provided by Keycloak:

- **User Identity Management:** Keycloak manages user identities, including registration, authentication, and profile management.
- **Authentication Mechanisms:** Secure authentication protocols, such as OAuth and OpenID Connect, are implemented through Keycloak to verify user identities and ensure secure access to the platform.
- **Authorization Policies:** Keycloak defines and enforces authorization policies that determine what actions users can perform and what resources they can access based on their roles and permissions.
- **Role-Based Access Control (RBAC):** Keycloak assigns roles to users and manages permissions based on these roles to ensure appropriate levels of access.
- **Single Sign-On (SSO):** Keycloak enables users to log in once and gain access to multiple related but independent systems without re-authenticating.
- **Multi-Factor Authentication (MFA):** Keycloak supports adding an additional layer of security by requiring users to provide two or more verification factors to gain access.
- **Audit Logging:** Keycloak provides audit logging of authentication and authorization activities to monitor for suspicious behaviour and ensure compliance with security policies.

3.7.3 Integration Points

The Authentication and Authorization component integrates with multiple other components within the CopernicusLAC platform to provide a cohesive and secure user experience:

- **User Workspace:** Ensures that only authorised users can access and manage their personalised workspace environments.
- **Front-end Services:** Controls access to web applications, interactive graphical tools, notebooks, and development environments based on user roles and permissions.

- Data Discovery and Access Component: Manages user access to EO data, ensuring that only authorised users can search, retrieve, and download data.
- Data Processing Component: Regulates access to processing services, allowing only authorised users to submit, monitor, and manage processing jobs.
- Application Registry: Controls access to the application packages, ensuring that only authorised users can manage and deploy applications.
- Resource Management: Coordinates with resource management to enforce security policies and access controls on provisioned resources.

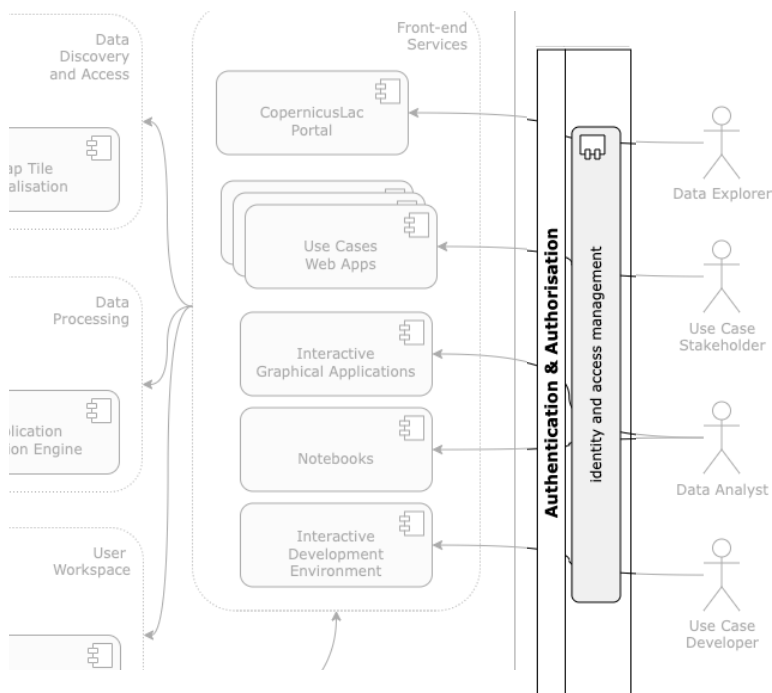


Figure 9 - Authentication and Authorization integration points

3.7.4 Software Stack

The Authentication and Authorization component utilises Keycloak for a comprehensive solution, eliminating redundancy and ensuring streamlined implementation:

- Keycloak: An open-source identity and access management solution providing single sign-on, identity brokering, user federation, authentication mechanisms (OAuth, OpenID Connect), role-based access control (RBAC), multi-factor authentication (MFA), and audit logging.
- OAuth and OpenID Connect: Integrated within Keycloak for secure authorization and authentication.
- LDAP Integration: Keycloak integrates with LDAP for managing user credentials and profiles.
- JWT (JSON Web Tokens): Used by Keycloak for securely transmitting user information during authentication and authorization processes.

- IAM (Identity and Access Management): Keycloak manages roles, permissions, and access policies across the platform.

3.8 Resource Management

3.8.1 Purpose

The Resource Management component of the CopernicusLAC platform is designed to provision, manage, and scale the infrastructure resources required by the platform's various components. This includes managing cloud resources, ensuring optimal utilisation, and maintaining high availability and reliability of services. The primary purpose of this component is to automate and streamline resource management tasks, reducing operational overhead and enabling the platform to scale dynamically based on demand.

3.8.2 Implementation

The implementation of the Resource Management component involves several key elements designed to provide automated management of resources, utilising Crossplane:

- Infrastructure Provisioning: Crossplane automates the provisioning of cloud infrastructure resources such as virtual machines, storage buckets, databases, and networking components.
- Resource Configuration: Crossplane manages the configuration of infrastructure resources, ensuring they are set up according to best practices and platform requirements.
- Dynamic Scaling: Crossplane supports dynamic scaling of resources based on predefined policies and real-time demand, ensuring the platform can handle varying workloads.
- Resource Monitoring: Continuous monitoring of resource utilisation and performance to ensure optimal operation and to proactively address any issues.
- Secret Management: The External Secrets Operator (ESO) is used to securely manage and inject secrets (e.g., API keys, passwords) into the resource configurations, ensuring sensitive information is handled securely.
- Policy Enforcement: Ensuring that all resource provisioning and management actions comply with defined policies and standards, enhancing security and compliance.

3.8.3 Integration Points

The Resource Management component integrates with multiple other components within the CopernicusLAC platform to provide a cohesive infrastructure management experience:

- User Workspace: Manages the provisioning of dedicated namespaces and resources within the Kubernetes cluster for individual users, ensuring isolated and secure work environments.

- **Front-end Services:** Instantiates and configures the necessary resources for the deployment and operation of front-end services, such as web servers and load balancers.
- **Data Discovery and Access Component:** Ensures the necessary storage and compute resources are available for data ingestion, cataloguing, and retrieval processes.
- **Data Processing Component:** Provisions the compute resources required for executing data processing workflows and handling large-scale EO data processing tasks.
- **Application Registry:** Manages the infrastructure resources needed for storing and retrieving application packages and container images.
- **Authentication and Authorization:** Coordinates with security components to enforce access controls and manage the infrastructure supporting identity and access management services.

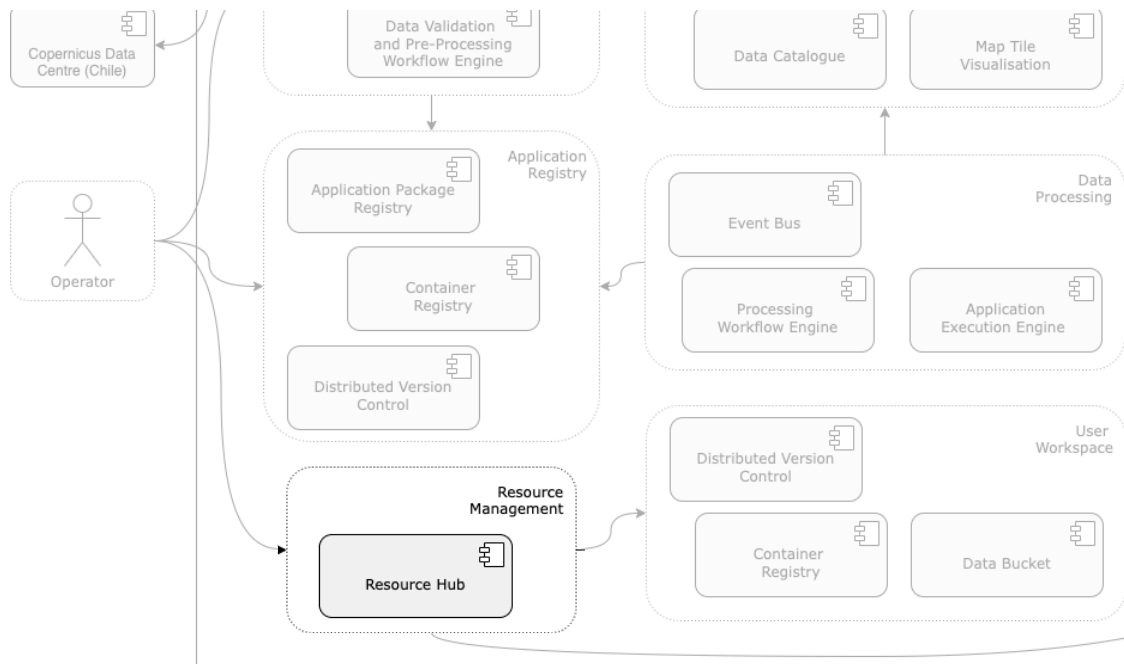


Figure 10 - Resource Management integration points

3.8.4 Software Stack

The Resource Management component utilises a range of software tools and technologies to achieve its objectives:

- **Crossplane:** An open-source control plane that enables the provisioning and management of cloud infrastructure resources through Kubernetes-native APIs.
- **Kubernetes:** Used to orchestrate and manage containerized applications and infrastructure resources, providing scalability and reliability.

- External Secrets Operator (ESO): Manages and injects secrets into Kubernetes resources, ensuring secure handling of sensitive information.
- Terraform: Optionally used for defining and provisioning infrastructure as code, complementing Crossplane's capabilities.
- Prometheus and Grafana: For monitoring resource utilisation and performance metrics, providing visibility into the health and efficiency of the infrastructure.
- Helm: Manages the deployment and configuration of Kubernetes applications, ensuring consistent and repeatable setups.
- Cloud Provider APIs: Interfaces with cloud service providers (e.g., AWS, Azure, GCP) to provision and manage infrastructure resources.

4 Data Ingestion Process

The purpose of this section is to describe the process by which EO data is integrated into the CopernicusLAC platform. This process is essential for ensuring that data is accurately brought into the system, validated, and made available for subsequent discovery and processing activities.

Data ingestion is the process of collecting, validating, and storing EO data. This process ensures that data is properly formatted, compliant with metadata standards, and securely stored for further use. The ingestion process might also involve pre-processing steps such as data or format transformation to ensure compatibility with the platform's storage and processing systems.

The section outlines the technologies, workflows, and standards involved in data ingestion, ensuring consistency and reliability across the platform.

4.1 Technologies Used

The data ingestion process in the CopernicusLAC platform relies on several key technologies that enable flexible and efficient workflows, accommodating different levels of ingestion, metadata registration, dataset copying, and dataset calibration. These technologies ensure that the platform can adapt dynamically to the specific requirements of each dataset and operational scenario:

- **Common Workflow Language (CWL):** CWL is used to define the workflows that automate data ingestion tasks. This includes specifying how data is retrieved, validated, and processed based on the ingestion level:
 - For metadata registration, CWL workflows focus on retrieving and validating metadata for cataloging.
 - For dataset copying, CWL workflows include tasks for downloading, integrity checks, and storing the datasets locally.
 - For calibration workflows, additional steps for processing and transformation, such as format conversion or quality enhancements, are included.
- **Argo Workflows:** Argo orchestrates the ingestion pipelines, ensuring that each step in the process is executed in the correct order and that dependencies are managed effectively. Its Kubernetes-native design provides the scalability required to handle the dynamic demands of different ingestion levels. For example:
 - Metadata-only ingestion workflows are lightweight and emphasize speed and compliance.
 - Full dataset ingestion workflows manage larger data volumes and include validation for data integrity and completeness.
 - Calibration workflows leverage Argo's flexibility to include custom processing tasks tailored to specific analytical needs.

- **STAC (SpatioTemporal Asset Catalog):** STAC is used to organise and catalog all ingested data, regardless of its ingestion level. Metadata from all ingestion workflows is standardized and indexed to ensure discoverability and accessibility through the platform. For metadata registration, STAC entries point to external data sources, while for dataset copying and calibration, the entries include details about locally stored datasets or pre-processed outputs..

4.2 Ingestion Workflow

The following diagram illustrates the overall data ingestion workflow, detailing the processes involved in handling metadata-only ingestion, dataset copying, and dataset calibration. These workflows are dynamically tailored based on the specific requirements of each dataset, ensuring efficient and relevant data handling.

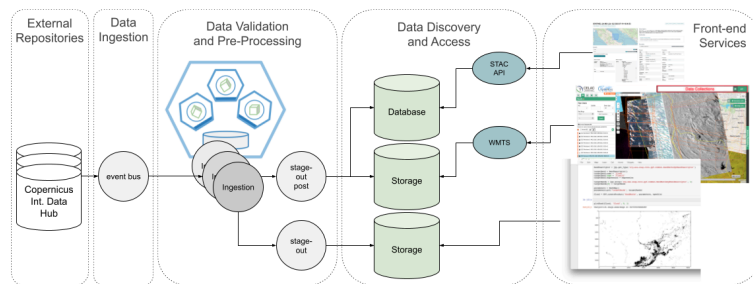


Figure 11 - Ingestion Workflow Diagram

The ingestion workflow involves several key stages that adapt to the selected ingestion level:

1. **Format Assertion:** The process begins with ensuring that the ingested data, whether metadata or full datasets, adheres to the platform's accepted formats. For metadata-only ingestion, the platform validates the format of metadata files to ensure compatibility with the STAC specification. For datasets requiring copying or calibration, the platform checks for supported formats (e.g., GeoTIFF, Cloud-Optimized GeoTIFF, or .zip) to maintain standardization. Datasets or metadata that fail to meet these requirements are filtered out at this stage to prevent workflow disruptions.
2. **Ingestion Application Package Selection:** Based on the dataset's ingestion level, the platform selects the appropriate Application Package. This package defines the specific Extract, Transform, Load (ETL) process to be applied. For metadata-only ingestion, the package includes tools for validating metadata and registering it in the catalog. For dataset copying, the ETL process involves additional steps for downloading, validating, and storing the dataset. Calibration workflows include specialized processing tools for transformations such as georeferencing, spatial adjustments, or format conversion. Selecting the correct package ensures the data is processed using optimized methods tailored to its ingestion level.

3. **Data Validation and Pre-Processing:** Validation and pre-processing steps vary based on the ingestion level:
 - a. Metadata-only ingestion: Focuses on metadata integrity, structure, and compliance with the STAC specification.
 - b. Dataset copying: Includes validation of data integrity through checksum verification and structural checks to confirm completeness.
 - c. Calibration workflows: Apply content-specific validations and transformations, such as aligning datasets to a standardized spatial reference or ensuring temporal consistency. These steps ensure that all data meets the platform's quality standards for further use.
4. **Metadata Extraction:** Metadata is extracted and organised according to the STAC specification for all ingestion levels. For metadata-only ingestion, this step finalizes the registration of metadata and its integration into the platform's catalog. For dataset copying and calibration, metadata extraction includes capturing details of the original dataset alongside transformations applied during ingestion.
5. **Uploaded Data Enrichment:** For datasets requiring additional functionality, such as advanced visualization or analytical tools, enrichment processes are applied. . These processes may include generating new COGs or defining rendering configurations for data visualisation. Metadata-only ingestion typically skips this step, while dataset copying and calibration workflows implement it as needed to enhance usability..
6. **Catalog Registry:** Regardless of the ingestion level, the generated STAC items and their associated dataset or metadata are registered in the platform's catalog. This registration ensures that all ingested data is discoverable and accessible to users, forming an essential part of the platform's data ecosystem. Metadata entries for externally hosted datasets include pointers to their access points, while locally stored datasets include references to their physical location.
7. **Asset Loading and S3 Storage:** For ingestion workflows involving dataset copying or calibration, the validated and processed data content is stored in the platform's S3-compatible object storage system. This storage solution is crucial for ensuring that data is readily available for discovery, visualisation, and further processing. For metadata-only ingestion, no physical storage occurs beyond catalog registration, maintaining storage efficiency.

To ensure the reliability of the data ingestion process, error handling mechanisms are in place. If an error occurs during data retrieval, validation, or storage, the system logs the error and triggers predefined recovery procedures. For example:

- Metadata-only ingestion: Flags incomplete metadata for manual review.
- Dataset copying: Retries download or validation steps, and if unresolved, alerts system administrators.
- Calibration workflows: Isolates and logs problematic data points, preventing disruption to the larger workflow.

By leveraging these technologies and following this workflow, the CopernicusLAC platform ensures that EO data is ingested, validated thoroughly, and stored securely, laying the foundation for effective data discovery and processing across the platform.

4.3 Application Package Selection

The selection of the Ingestion Application Package is an initial step to have the right ETL process in the data ingestion process. This selection determines how the data will be processed, ensuring that the appropriate tools and workflows are applied to each dataset based on its specific characteristics.

- **Purpose:** The purpose of selecting an Ingestion Application Package is to tailor the data ingestion process to the specific requirements of the dataset. By selecting the correct package, the platform ensures that data is processed using the most suitable methods, optimising quality, consistency, and usability.
- **Implementation:** The platform uses predefined rules and metadata properties (e.g., sensor type, data format, mission-specific characteristics) to match each dataset with the appropriate Ingestion Application Package. These packages are encoded using the CWL and may include specific tools, software dependencies, and processing steps that are required for the data type.
- **Customization:** If the dataset has unique properties not covered by existing packages, a new Application Package can be created and deployed. This allows for flexibility in handling a wide variety of EO data types and ensures that the ingestion process remains robust and adaptable.
- **Integration with Argo Workflows:** Once selected, the Ingestion Application Package is executed as part of the Argo Workflows pipeline, which orchestrates the ingestion, validation, and storage processes. This integration ensures that the workflow is executed from data upload to final storage.

The application package selection involves choosing the appropriate ETL process that is validated to act on the uploaded data based on the properties of the STAC Item. This is done by using predefined configuration items within the STAC properties to match the dataset with the corresponding ETL process, ensuring that the data is properly processed. If the STAC Item lacks specific properties, a generic ETL process is employed, which may result in limited discovery and visualisation functionalities. This targeted selection ensures that each dataset undergoes the most suitable transformation and enrichment process for optimal usability.

The Ingestion ETL processes are implemented as OGC Earth Observation Application Packages encoded using the Common Workflow Language (CWL) and include necessary tools and workflows. The Application Package reads a STAC Catalog that references the uploaded STAC Item and its assets, transforms them as needed, and writes a new STAC Catalog. The transformed STAC Item and its assets are published in the object storage and registered in the catalogue for discovery and access.

4.4 Data Validation and Content Validation

After the application package has been selected, the data undergoes a content validation step tailored to the chosen ingestion level. This ensures the integrity, compliance, and usability of the ingested data, whether it involves metadata-only registration, dataset copying, or dataset calibration. The validation process dynamically adjusts to the ingestion level to ensure efficient handling of data requirements.

For metadata-only ingestion workflows, validation focuses on the compliance and completeness of metadata records. Key steps include:

- **STAC Item Presence:** The dataset must include a valid STAC Item at the root level of the metadata archive. This ensures the metadata conforms to the cataloging standards required for discovery.
- **STAC Item Validation:** The STAC Item is checked for compliance with the STAC specification, including mandatory fields (e.g., id, type, bbox, datetime) and valid data formats. Deviations are flagged for manual review to maintain the integrity of the metadata catalog.

For workflows involving dataset copying, validation extends to the content of the datasets themselves, ensuring their readiness for storage and access. Key steps include:

- **STAC Asset Presence:** The metadata must reference all associated data files, ensuring that the dataset includes all necessary components. Missing references result in the dataset being flagged for further review.
- **Checksum Verification:** The platform calculates and verifies the checksums of all referenced assets to ensure data integrity. Any inconsistencies trigger an error, and the dataset is moved to temporary storage for manual inspection.
- **Format Validation:** Datasets are checked to confirm that they adhere to supported formats (e.g., GeoTIFF, COG, or .zip), ensuring compatibility with downstream workflows.

For workflows involving dataset calibration, validation includes additional checks and transformations to ensure that the datasets meet predefined analytical or quality requirements:

- **Calibration-Specific Checks:** The content is evaluated for parameters relevant to the calibration workflow, such as spatial resolution, temporal alignment, or sensor-specific attributes. Any deviations from the required specifications are flagged for adjustment or operator intervention.
- **Transformation Validation:** After calibration processes such as georeferencing, re-projection, or resampling, the datasets are validated to confirm that the transformations have been applied correctly and do not introduce errors.
- **Enhanced Metadata Validation:** The metadata is updated to reflect the calibration processes applied, ensuring traceability and compatibility with analytical workflows.

For all ingestion levels, the platform ensures compliance with overarching standards and best practices:

- **STAC Item Validation:** Regardless of ingestion level, the STAC Item is validated for mandatory fields, appropriate links to data assets, and correct formatting. This guarantees interoperability with the platform's catalog and discovery tools.
- **Error Handling:** If any validation step fails, the platform logs the error and isolates the dataset or metadata in temporary storage. Depending on the issue, the system may retry the process, flag the dataset for manual inspection, or alert system administrators for resolution.

4.5 Metadata Profile

CopernicusLAC will encompass a wide spectrum of technical operations, from ingesting EO products to generating data from various processing tasks. Given this broad scope, it's essential to have a streamlined metadata approach. The minimal metadata profile, based on the STAC standard, offers a consistent description format that enhances data discovery, analysis, and management across all operational stages.

This profile serves both ingested and processed EO products. It integrates key attributes and descriptors, ensuring every data point, be it raw, ingested, or an outcome of processing, comes with a standardised and clear metadata. By recognizing the unique needs of both input and output datasets, the profile offers a unified yet adaptable metadata structure for all geospatial assets in CopernicusLAC.

An important component of the data ingestion process is the application of a minimal metadata profile to ensure consistency and interoperability. This profile defines the essential metadata fields required for each dataset and is stored in accordance with the STAC specification, enabling integration with the platform's data discovery and access components.

Table 1 shows a minimal metadata profile using STAC that defines a subset of mandatory fields that every geospatial asset should have. This definition will ensure consistency in metadata across all datasets while also aligning with the STAC specification.

Table 1 - Minimal Metadata Profile

Name	Type	Description
id	string	A unique identifier for the item.
type	string	The nature of the STAC item, i.e. "Feature".
stac_version	string	Specifies the STAC version to which the item adheres, e.g., "1.0.0".

stac_extensions	Array of strings	An array listing the STAC extensions the item implements. Must include EO extension.
geometry	object	Contains the spatial footprint of the item. Uses the GeoJSON Geometry format.
bbox	Array of floats	Bounding box array for the item in the WGS84 coordinate reference system. Format: [min lon, min lat, max lon, max lat].
properties	object	An object encapsulating descriptive properties about the item. Examples of properties included are detailed below.
- datetime	string	A string, formatted as a date-time, representing the date and time of the asset capture.
links	Array of Link objects	An array containing Link objects. This must include a self-link pointing to the item's own URL and a link to its parent collection.
assets	object	An object that lists asset objects available for download, each uniquely keyed. Examples of assets included are detailed below.
- thumbnail	asset object	An asset object linking to a small preview image for quick visualisation.
- data	asset object	An asset object linking to the primary geospatial data file(s).
collection	string	A string denoting the STAC Collection ID to which this item belongs.

4.6 Metadata Enrichment for Visualization

To facilitate visualisation of the dataset in the full scope of the platform services, the ingestion process leverages STAC extensions to provide rendering configurations and web map links for visualisation services.

The process leverages the Rendering² STAC Extension to define rendering configurations, such as creating true-colour or colour-infrared composites from different bands. If the STAC Item and its assets are already in the COG format and include rendering information, the dedicated and collection specific ETL process focuses on generating Web Map Links³ using the Web Map Links STAC Extension. This allows for the integration with web map services like OGC WMTS and XYZ, providing end-users with access to visual representations of the data.

1. Visualisation STAC Extensions:
 - a. Rendering Extension: This extension provides details on how the STAC Item can be visualised (e.g., combining different bands to create a composite image).

² <https://github.com/stac-extensions/render>

³ <https://github.com/stac-extensions/web-map-links>

- b. Web Map Links Extension: This extension provides links to web maps (e.g., OGC WMTS and XYZ) that can render the STAC Item using the specified visualisation configuration.
2. Enrichment Process:
 - a. Existing Visualisation Information: If the STAC Item already includes rendering information, the ETL process adds Web Map Links URL templates to implement these configurations.
 - b. Generating New Assets: For datasets that do not have visualisation-ready assets, the collection-specific / or a dedicated ETL process generates new COG and updates the STAC Item with rendering information and Web Map Links.
 - c. Publication: The enriched STAC Item and new assets are published to the object storage and registered in the catalog.

An important aspect of the ETL process is the enrichment of data to enable visualisation. This involves leveraging the Rendering STAC Extension and the Web Map Links STAC Extension:

- Rendering Extension: Provides rendering configurations for the STAC Item, such as combining different bands to create true-colour or colour-infrared composites.
- Web Map Links Extension: Supplies links to web maps (e.g., OGC WMTS and XYZ) that can render the STAC Item using the specified visualisation configuration.

If the STAC Item included in the .zip archive contains the Rendering extension and the assets are in the Cloud Optimized Geotiff (COG) format, the ETL process focuses on generating the Web Map Links.

Here is an example of a STAC Item with rendering information:

```
"renders": {  
  "trc": {  
    "title": "True-colour composite",  
    "assets": [  
      "red", "green", "blue"  
    ],  
    "resampling": "nearest",  
    "rescale": [  
      [0, 5000], [0, 5000], [0, 5000]  
    ]  
  },  
  "civ": {  
    "title": "Colour-infrared composite",  
    "assets": [  
      "red", "green", "nir"  
    ]  
  }  
}
```

```
],  
  "resampling": "nearest",  
  "rescale": [  
    [0, 5000], [0, 5000], [0, 5000]  
  ]  
}  
}
```

And the web map links as an output of the ETL:

```
{  
  "rel": "xyz",  
  "href":  
    "https://view.imagery.org?assets=red&assets=green&assets=blue&resampling=  
    =nearest&rescale=[0,5000]&rescale=[0,5000]&rescale=[0,5000]&",  
  "type": "image/png",  
  "title": "True-colour composite",  
  "render": "trc"  
},  
{  
  "rel": "xyz",  
  "href":  
    "https://view.imagery.org?assets=red&assets=green&assets=nir&resampling=  
    nearest&rescale=[0,5000]&rescale=[0,5000]&rescale=[0,5000]&",  
  "type": "image/png",  
  "title": "Colour-infrared composite",  
  "render": "civ"  
}
```

5 Data Discovery, Access and Visualisation

Data discovery in the CopernicusLAC platform is designed to be comprehensive and user-friendly, ensuring that users can locate and retrieve EO data relevant to their needs. The process is supported by metadata cataloguing, indexing, and an intuitive interface that allows users to interact with the data in meaningful ways. Central to this design is the integration of the STAC and the STAC API, which provide a standardised framework for describing and accessing EO data.

5.1 SpatioTemporal Asset Catalog (STAC)

STAC serves as a standardised representation for all these geospatial assets and their associated metadata. Metadata, in this context, includes information about when and where a dataset was collected, the type of sensor used, its resolution, and other descriptive details. By adhering to a common specification provided by STAC, various organisations and platforms are representing their geospatial assets in a manner that's consistent and easily comprehensible. This uniformity ensures that different datasets, even if they are from diverse sources or missions, can be understood and compared with minimal friction in a federated landscape.

One of the defining features of STAC that is extremely relevant for CopernicusLAC is its flat-file nature, meaning it is stored alongside the data it describes, usually as a JSON file. This co-location ensures that the metadata remains directly accessible and linked to its corresponding dataset. Within a STAC item, individual assets are detailed, representing various components or layers of a geospatial product, such as RGB images, thermal bands, or other data files. By defining these assets, STAC provides users with a comprehensive view of what each dataset comprises, aiding in precise data retrievals.

The STAC API optimises geospatial data accessibility with a dynamic interface for users and systems to search and access the catalogued assets. Adhering to modern RESTful principles, the API facilitates interactions with STAC Items and Collections. By using this interface, users can perform real-time searches, refining their queries based on spatiotemporal parameters, specific asset types, or even custom metadata fields. This dramatically reduces the time taken to locate pertinent datasets, streamlining the entire data discovery process.

In this context, STAC ensures that geospatial data can be easily catalogued, discovered, and accessed in a standardised manner by adhering to certain conventions and rules:

- **STAC Item:** The core building block of STAC, a STAC Item represents a spatiotemporal asset and contains a collection of metadata. Each item is described using JSON, a lightweight data-interchange format.
- **Assets:** Within each STAC Item, assets define the actual data files. They might refer to different bands of satellite imagery, resolution levels, or even derived products.

Assets specify URIs for data retrieval, as well as media type identifiers to convey the nature of the data.

- **STAC Collection:** This is a collection of STAC Items, with metadata about the aggregation of items, potentially describing properties like the spatial and temporal extents of all the contained items.
- **Core and Extensions:** STAC is designed to be extensible. While it has a core specification that ensures basic functionality and interoperability, extensions can be added to cater to specific use-cases, like satellite, label, or point cloud extensions.
- **Linking Mechanism:** STAC uses a consistent linking mechanism. Every STAC item, catalog, and collection contains an array of links, which can be used to navigate to related items, parent or child catalogues, or even out to other specifications.
- **Spatial and Temporal Bounding Boxes:** Each STAC item provides spatial bounds (usually described by a bounding box or polygon) and temporal bounds (start and end times) to aid in discovery based on space and time criteria.
- **Flat-File Nature:** As previously mentioned, STAC specifications are often stored as flat JSON files alongside the data they describe. This co-location ensures rapid access to metadata without necessitating complex database queries.
- **Self-Describing JSON:** STAC relies on self-describing JSON structures, meaning each file is intended to be understood in isolation. This makes it scalable and easy to parse without deep dependencies.

A primary advantage of STAC is that it defines an extensible vocabulary, ensuring metadata consistency across varied datasets. Given the diverse sources of EO data, from an assortment of sensors, platforms, and missions, and ever-growing use cases, there's a potential for metadata variations. STAC's framework compels providers to use a harmonised set of conventions.

In addition to the core functionalities of STAC, the catalogue incorporates the "Web Map Links Extension"⁴. This extension enriches the STAC catalog by adding an additional link element within the STAC item structure.

The link elements provided together in the STAC Item points to web map services that are capable of visualising the geospatial data described by the item. It allows client applications to directly access and visualise the spatial data through interactive map services, enhancing their understanding and interpretation of the data.

To bolster the security and controlled access to the geospatial data the catalog also integrates the use of signed URLs through the STAC "Authentication Extension"⁵. The STAC Authentication Extension's signed URL information is specifically integrated into the STAC for secure data downloading. This mechanism ensures that access to spatial data assets is tightly controlled:

⁴ <https://github.com/stac-extensions/web-map-links>

⁵ <https://github.com/stac-extensions/authentication>

- **Signed URL Generation:** When a user requests access to an asset, the system generates a signed URL. This URL is uniquely created for the specific request and user, embedding security credentials directly into the URL.
- **Temporary Access:** Each signed URL is valid for a limited time, ensuring that access to the asset is temporary and controlled. This time-bound nature of URLs enhances security by minimising the risk of unauthorised access.
- **Direct and Secure Data Download:** The signed URLs allow users to download assets directly from the catalog without the need for additional layers of authentication at the point of download, streamlining the user experience while maintaining security.

5.2 Catalogue API

The platform utilises the SpatioTemporal Asset Catalog (STAC) API to optimise geospatial data accessibility. The API functions as a dynamic interface for users and systems to search and access the assets catalogued by STAC. Adhering to modern RESTful principles, the API facilitates interactions with STAC Items and Collections. By using this interface, users can perform real-time searches, refining their queries based on spatiotemporal parameters, specific asset types, or even custom metadata fields. It dramatically reduces the time taken to locate pertinent datasets, streamlining the entire data discovery process.

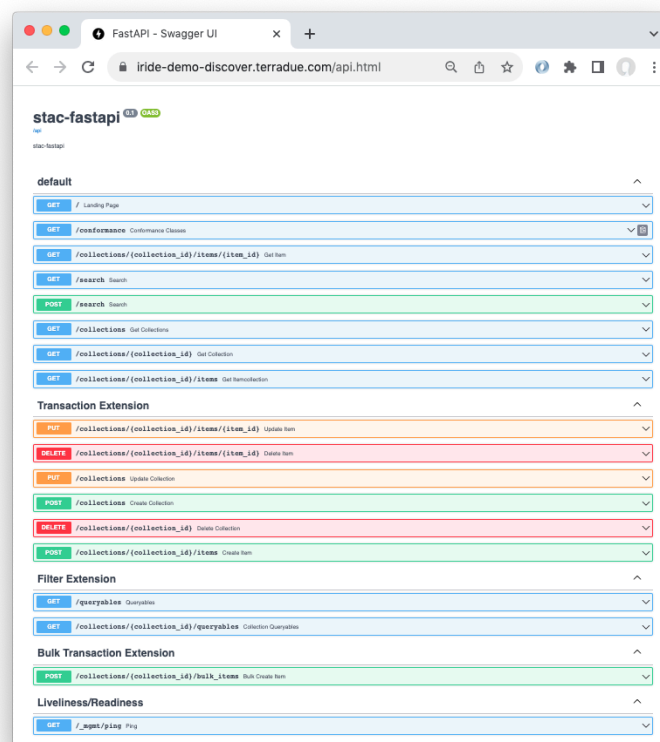


Figure 13 - STAC FastAPI Interface

Here are some technical examples of how these queries can be structured:

- **Temporal Queries:** The STAC API allows users to search for assets within specific date and time ranges. For instance, a user can retrieve all assets captured between January 1, 2020, and December 31, 2020, using a query like:

```
GET /stac/search?datetime=2020-01-01T00:00:00Z/2020-12-31T23:59:59Z
```

- **Spatial Queries:** Through the API, spatial extents can be defined using bounding boxes or polygons. A user could, for instance, search for all assets that cover a specific city using its bounding box coordinates:

```
GET /stac/search?bbox=-0.15,51.4,-0.1,51.6
```

- **Cloud Cover:** Leveraging the extended capabilities of STAC, users can even filter assets based on atmospheric conditions. For instance, one could query for assets with a cloud cover of less than 20% like:

```
GET /stac/search?query=eo:cloud_cover<20
```

- **Satellite Mission Specific Search:** To retrieve images only from a specific collection representing a specific mission or sensors, say Sentinel-2:

```
GET /stac/search?collections=[sentinel-2]
```

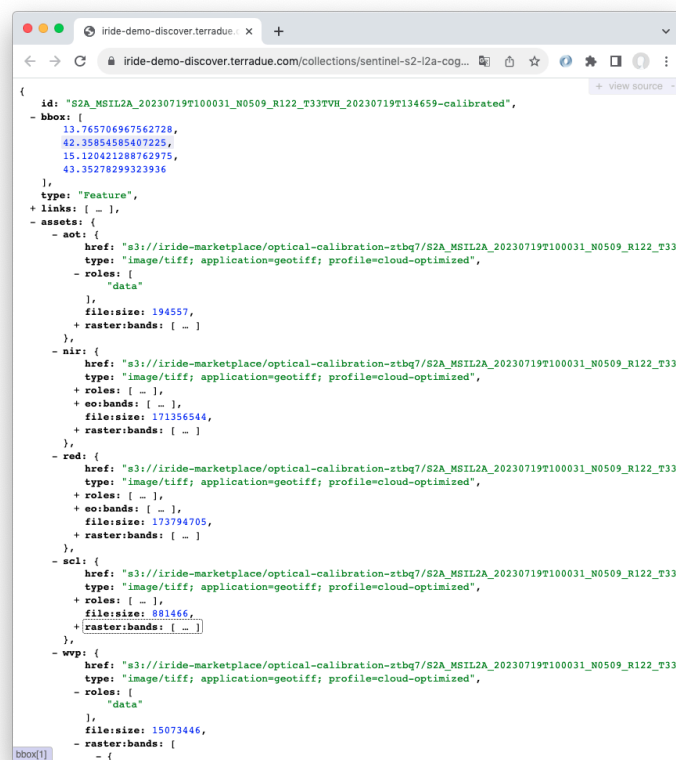


Figure 14 - JSON response from a STAC API query

Furthermore, the true power of STAC API emerges when these queries are combined. As an example, a combined query could look for Sentinel-2 assets, over a specific region captured during 2020:

```
GET /stac/search?collections=[sentinel-2]&bbox=-0.15,51.4,-0.1,51.6&
datetime=2020-01-01T00:00:00Z/2020-12-31T23:59:59Z
```

The STAC API also leverages the Common Query Language (CQL) for advanced querying capabilities submitted as POST requests to the API. CQL is a query language standard by the Open Geospatial Consortium (OGC) designed for filtering vector features. When used in conjunction with STAC, CQL provides a powerful and expressive way to search for geospatial assets based on their metadata attributes. Using CQL with STAC API enhances the flexibility and precision of queries. For example, rather than just relying on simple key-value pair queries, with CQL you can incorporate logical, arithmetic, temporal, and spatial operators to form more complex search criteria.

Beyond just enabling search functionalities, the STAC API provides mechanisms for dynamic data filtering, pagination, and sorting, all tailored to present the most relevant results to a user's specific requirements. Furthermore, the inherent modularity of the STAC design allows for the definition and support of new extensions, ensuring adaptability to address emerging and specialised use cases. As the catalog grows and the use cases addressed by the platform evolve, the STAC API remains agile. This ensures consistent delivery of optimum performance, guaranteeing that users always have the best tools at their disposal for data discovery and access.

The Catalogue API can be further enhanced by integrating the STAC API - "Filter Extension Specification"⁶. This extension augments the capabilities of our Catalogue API, providing advanced filtering options for querying geospatial data.

This extension allows client applications to perform more complex queries on the catalog data. It supports filtering by various attributes such as spatial dimensions, time ranges, and specific properties of the geospatial data. This enables users to precisely locate the data that meets their specific criteria. By incorporating this extension, users can easily navigate through large datasets and find relevant geospatial assets. This not only improves the usability of the Catalogue API but also significantly reduces the time and effort needed to find the right datasets.

5.3 Data Access and Retrieval

The inherent characteristics of STAC and COG mean that we have the capability to directly port data into subsequent fully native cloud-based applications. STAC offers a structured approach to cataloguing geospatial assets and the STAC file, primarily consisting of

⁶ <https://github.com/stac-api-extensions/filter>

standardised metadata and pointers to data assets, simplifies data processing flow management. This becomes invaluable in multi-stage EO data processing workflows as it circumvents the traditionally cumbersome process that necessitated the sequential downloading, transformation, and subsequent uploading of datasets, a methodology that not only consumed significant time but also required substantial computational resources. Moreover, adopting the COG format introduces palpable economic advantages. Specifically, the format's design allows for access to only the indispensable segments of data, which effectively means that there's an elimination of any superfluous storage or transfer costs traditionally associated with handling extensive datasets.

When dealing with EO datasets, this means that users and applications can interactively access, visualise, and analyse specific sections of a geospatial raster directly from the cloud storage. This integration plays a pivotal role, especially in applications like real-time visualisation, on-the-fly analytics, and large-scale processing tasks where timely data access is paramount. Moreover, the synergy between STAC, COG, S3, and geospatial tools extends beyond just efficiency. It cultivates an environment where data is more accessible, shareable, and versatile. With COG support, results of EO products processing can be utilised by a broad spectrum of third-party applications, fostering collaboration and widening the horizons for data application.

STAC provides a standardised approach to data referencing, regardless of the processing model, ensures efficiency, accuracy, and scalability in EO data processing workflows. For example, in an application processing sequence that involves image rectification, merging, and change detection, a STAC file serves as a structured manifest. It consistently points to the necessary data assets for each step, ensuring that processing occurs on the appropriate datasets. The metadata within the STAC file provides specifics like timestamps, sensor details, and geographical bounds. This enables applications to programmatically select, filter, or prioritise data during processing. By employing STAC files, EO data processing applications can achieve streamlined data access, uniform representation, and efficient processing, without navigating through the complexities of diverse and dispersed geospatial datasets.

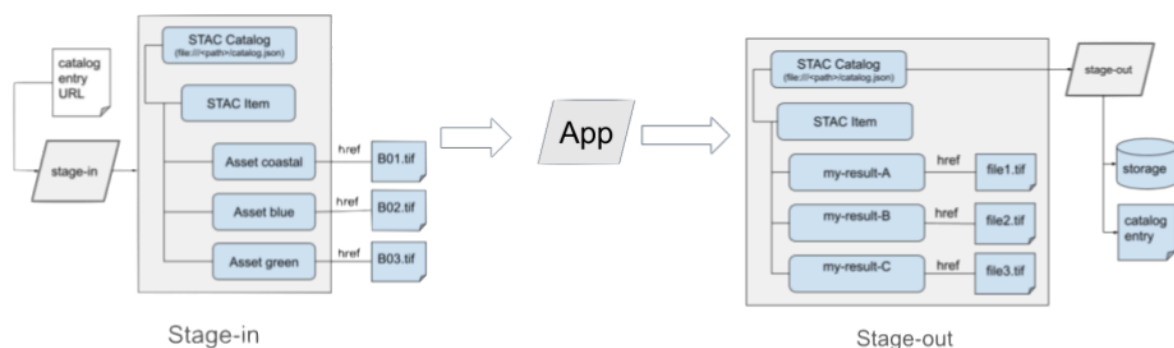


Figure 15 - STAC-Enabled EO Data Processing Workflow

Figure above illustrates the role of STAC in standardising the referencing of EO data throughout various stages of an application's processing workflow. At the 'stage-in' phase, the process begins with a STAC Catalog, which is essentially a JSON file containing metadata and links to STAC Items representing EO data assets. These assets, such as spectral bands 'coastal', 'blue', and 'green', have their respective access references which point to their physical locations.

As the data transitions to the application, the STAC Catalog ensures a structured and orderly reference to all necessary data assets. This setup allows the application to process the EO data, whether it involves rectification, merging, change detection, or any other geospatial analysis task. The application, informed by the metadata within the STAC file, can programmatically filter and process the data according to timestamps, sensor specifications, and geographic boundaries.

Upon processing completion, the workflow moves to the 'stage-out' phase, where the application generates new STAC Items. These items now point to the processed results which are then stored for future access or distribution.

This workflow demonstrates how STAC provides a consistent and organised approach to handling EO data, ensuring that applications can access, utilise, and generate processed data without the need to manage varied data formats and structures manually. The use of STAC significantly enhances the efficiency, accuracy, and scalability of EO data processing workflows, catering to the needs of a wide range of geospatial applications within the CopernicusLAC initiative.

Also for managing and guiding the flow of data through various processing paradigms STAC Catalog files can be instrumental in creating manifests for both fan-in and fan-out processing instances. In a fan-in scenario, where multiple datasets are aggregated for a singular processing task, the STAC Catalog file can enlist all the necessary assets, ensuring that all input datasets are uniformly represented and accessible. Conversely, in a fan-out situation, where an individual dataset may lead to multiple parallel processing tasks, the STAC Catalog file ensures that each processing task has a clear and consistent reference to the correct subset of data.

5.4 Graphical User Interface

The platform is designed with a dual-interface approach to data discovery, ensuring that users have access to the data they need in a way that suits their preferences and technical proficiency. At the core of this approach is the integration of a user-friendly graphical user interface (GUI) along with existent APIs.

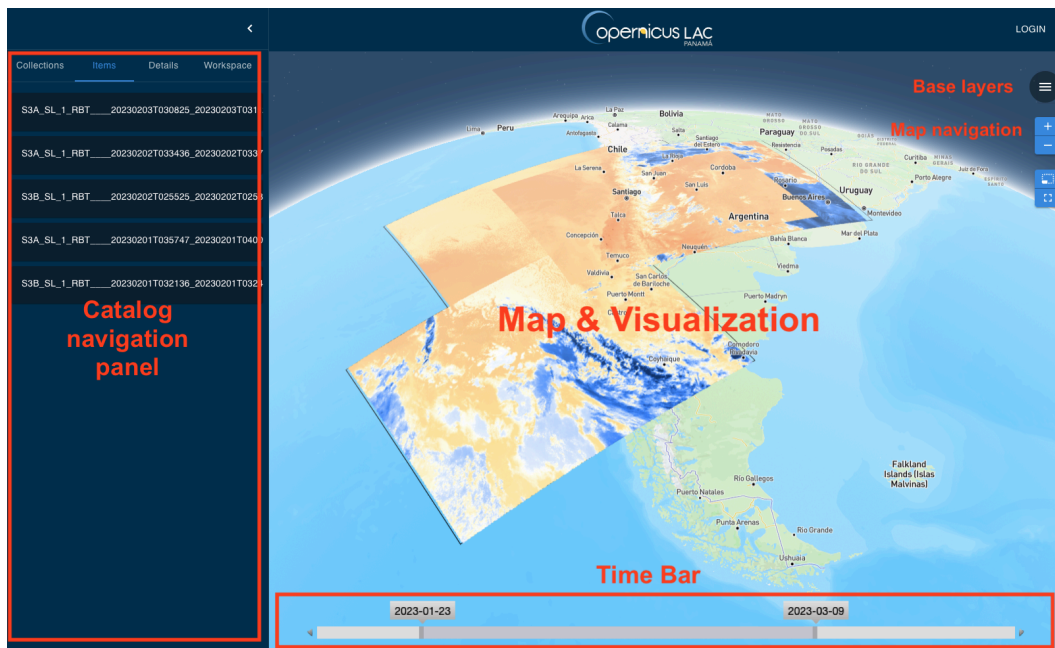


Figure 16 - Generic Schema of Discovery and GUI.

As seen in the diagram of the previous figure, the GUI is organised to provide an intuitive and navigable experience for all users, regardless of their technical background. It will be the visual gateway to the array of CopernicusLAC data, allowing users to search, view, and download datasets through a web-based portal. The design of the GUI will focus on ease of use, with clear categorizations, filters, and search capabilities that allow users to quickly find the data they require. It is here that users can benefit from a visual representation of search results, interact with maps, and understand the spatial and temporal context of the data before downloading or integrating it into their workflows.

The GUI for data discovery and visualisation system will reuse existing components to offer an intuitive and comprehensive experience for the user with the following components::

- **Search & Results:** This area is dedicated to allowing users to conduct searches using various criteria. Users can input keywords, use advanced filters, or navigate through pre-set categories to find the specific datasets they need. Once a search is initiated, this section will display the results in a clear and organised manner, enabling users to quickly identify the most relevant data.
- **Navigation & Search Toolbar:** Situated prominently, this toolbar provides the necessary tools to refine search results. It includes features for adjusting search parameters, zooming in and out of result sets, and selecting specific data entries for more detailed views.
- **Map & Visualization:** At the heart of the interface is a dynamic map that visualises the geographic distribution of search results. Users can interact with the map to explore different regions, zoom into areas of interest, and click on individual data

points to reveal more information. This visual context is crucial for users to understand the geographical relevance of the data they are examining.

- **Time Bar:** This interactive timeline allows users to filter search results based on temporal data. Users can slide through time, identify data availability in different periods, and select specific time frames to narrow down their search to the most pertinent datasets.
- **Data Collections:** This section of the interface presents a structured view of the various data collections available within the CopernicusLAC framework. Users can browse through different collections, understand the scope of each dataset, and choose collections that align with their needs.
- **Processing:** This dedicated space is where users can access tools and services to process the data they have discovered. It provides a direct link to the suite of processing services, enabling users to initiate various data processing jobs from the same interface.
- **Base Layers:** Here, users can toggle between different map base layers, allowing them to customise the map's appearance according to their preferences or specific requirements for the data they are working with.
- **Free Text Search:** A simple yet powerful feature, the free text search gives users the freedom to enter queries in their own words, enhancing the accessibility of the search functionality.

Each of these components is thoughtfully integrated into the GUI, ensuring that users have a productive experience from data discovery to data processing. The GUI is designed not only to be user-friendly but also powerful enough to handle complex searches and data visualisations, catering to a diverse range of user needs and skill levels.

The toolbar on the CopernicusLAC data discovery platform will be a central element designed to empower users with advanced filtering capabilities for their search experience. It's a versatile interface that allows users to narrow down collections of EO data to the most relevant datasets based on specific criteria. Here's a detailed overview:

- **Filter Criteria:** Located at the top of the left panel, the Filter Criteria section is designed for precision. Users can filter EO data based on parameters, including:
 - **Area of Interest (AOI):** Users can define a geographic area for which they need data, which could range from a local region to a broad expanse.
 - **Satellite:** This filter lets users select data from specific satellite missions, ensuring they get the precise type of data they need.
 - **Sensor Type:** Different sensors provide different types of data. Users can select among various sensor types to match their application requirements.
 - **Date Range:** By setting a start and end date, users can retrieve data from specific time periods when performing temporal analysis.
 - **Resolution:** This allows the selection of data with the desired level of detail, which is especially important for applications requiring high-resolution imagery.

- **Geographic Filters:** These filters enable users to refine their search geographically on the map through intuitive tools:
 - Pinpoint: For focusing on a single, specific location.
 - Bounding Box: To create a rectangular search area that covers the region of interest.
 - Polygons: Users can draw custom shapes to define their search areas with greater precision.
 - Well Known Text (WKT): For users who prefer to input geographic data as text, the WKT format is supported, translating text to geometry.
 - Geographic Names: A feature that turns location names into administrative boundary polygons, automatically defining an area of interest.
- **Advanced Metadata Filters:** For users with specific requirements, the toolbar provides filters that delve into the intricacies of metadata:
 - Orbit and Track Number: Useful for identifying datasets from specific orbital paths or imaging sequences.
 - Cloud Coverage: Essential for optical data, this filter helps avoid imagery with excessive cloud cover.
 - Platform and Instrument Name: For those who need data from a specific platform or instrument, this filter ensures that only relevant datasets appear in the search results.
 - Segregated Acquisition: An on/off toggle that helps users filter for datasets based on their acquisition mode, separating standard captures from those with unique parameters.
- **Search Functionality:** Beyond structured filters, the toolbar includes:
 - Free Text Search: Where users can enter keywords or phrases related to their data needs, providing a more human-centric approach to data discovery.

The toolbar's interface is intuitive yet comprehensive, designed to cater to a diverse user base, from researchers and analysts to application developers and decision-makers. It ensures that all users, regardless of their technical expertise, can effectively harness the wealth of EO data available through the CopernicusLAC platform.

5.5 Visualisation Integration

The visualisation of the data is also a crucial component, bridging the gap between raw data and actionable insights. To enhance this capability, Titiler is integrated to harness the flexibility of modern geospatial practices, particularly the Cloud-Optimised Geotiffs (CoG) format, to convert geospatial raster data into web-friendly map tiles, which are then used to display maps on web applications.

Titiler is a dynamic stateless tiling server designed for web mapping applications that allows on-the-fly generation of map tiles is particularly useful for handling various types of geospatial data, including satellite imagery and elevation models. One of the significant

features of Titiler is its compatibility with multiple raster data formats. This flexibility makes it a versatile tool for different geospatial data applications.

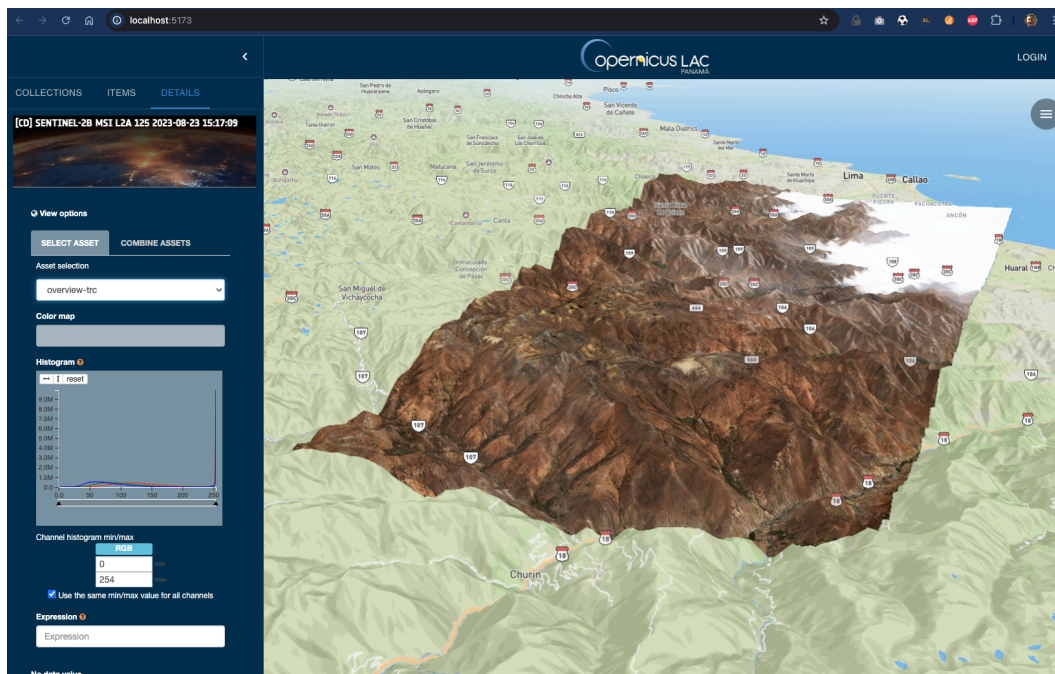


Figure 17 - Titiler-powered interface displaying a true colour composite and histogram

Titiler is optimised for cloud environments, especially cloud storage solutions like S3, which is a boon for managing large datasets common in geospatial analyses, and integrates with web map libraries such as Leaflet, OpenLayers, or Mapbox GL JS.

With Titiler, platforms can effortlessly generate tiles from large geospatial raster datasets directly from cloud storage, without the need to download or process the entire file. This server-based approach ensures high-speed visualisation even for massive datasets. Titiler capitalises on the innate advantages of CoG, where partial data reads can be performed, meaning that only the necessary portions of the data relevant to a user's viewport or area of interest are processed and displayed.

The integration of Titiler enables the creation of interactive and engaging web maps. While Titiler offers customization options for the tiles, including tile size and format, it manages to maintain a focus on simplicity and efficiency and, as an open-source project, Titiler benefits from community-driven development and improvements. This open-source nature not only makes it accessible for a wide range of users but also encourages continuous enhancements and adaptations to the evolving needs of web mapping.

6 Application Packaging

The purpose of this section is to outline the application packaging process within the CopernicusLAC platform. Application packaging is essential for ensuring that EO data processing applications are properly structured, versioned, and ready for deployment. This process supports consistency, scalability, and reliability across various processing tasks, allowing applications to be easily deployed and managed within the platform's infrastructure.

6.1 Overview

Application packaging in the CopernicusLAC platform is a systematic process that encapsulates all components necessary for EO data processing workflows. This includes the application code, dependencies, configuration files, metadata, and runtime environments. The primary objective is to create standardised, portable packages that can be deployed consistently across diverse environments, from local development setups to cloud-based infrastructures and to be deployed as a web service.

This process adheres to the Best Practices for Earth Observation Application Packaging⁷ as defined by the Open Geospatial Consortium (OGC) and the EO Exploitation Platform Common Architecture⁸ (EOEPCA) spearheaded by the European Space Agency (ESA). These guidelines ensure that EO applications are reproducible and portable across different execution scenarios.

The application packaging process leverages several key technologies:

- **CWL:** Standard for describing the workflows and tools used in EO data processing. It ensures that applications are portable and can be executed consistently.
- **GitLab:** GitLab is used for version control, managing the application code, workflows, and configuration files. It also handles Continuous Integration (CI) pipelines that automate the packaging process.
- **Harbor:** Harbor serves as the container registry where Docker images of the packaged applications are stored. These images include all necessary runtime environments and dependencies.
- **Kubernetes and Helm:** These tools are used to deploy the packaged applications within the platform's cloud environment. Kubernetes orchestrates the containers, while Helm manages the deployment configurations.

The packaging workflow involves several key steps:

- **Defining Workflows and Dependencies:** The first step involves defining the data processing workflows using CWL. This includes specifying tools, input/output files,

⁷ <https://docs.ogc.org/bp/20-089r1.html>

⁸ <https://eoePCA.org/>

and dependencies required by the application. CWL provides a standardised syntax that ensures workflows are portable and reproducible.

- **Creating Application Packages:** Application code, dependencies, and configuration files are packaged into a Docker image. This image is created using Dockerfiles, which specify the environment and dependencies required by the application.
- **Storing Packages in Harbor:** The Docker image, along with its metadata, is pushed to the Harbor container registry. Harbor ensures that these packages are securely stored and easily retrievable for deployment.
- **Versioning:** Each application package is versioned using GitLab. Version control allows developers to track changes over time, ensuring that different versions of an application can be maintained, tested, and deployed as needed.

6.2 Common Workflow Language (CWL)

The Common Workflow Language (CWL) offers a standardised and versatile approach to creating, sharing, and executing workflows. Developed to address the complexities of diverse computational environments, CWL provides a common ground for describing workflows, regardless of the underlying execution platform.

CWL's strength lies in its ability to facilitate the creation of executable workflows through a standardised syntax. For instance, imagine a scenario where an Earth observation researcher wants to process satellite imagery to calculate vegetation indices like NDVI. Using CWL, they can define the necessary steps, such as data loading, image preprocessing, and index calculation, in a single CWL document. This document acts as a blueprint that captures the sequence of operations, their inputs, outputs, and dependencies. This standardised approach not only simplifies the communication of workflows but also fosters collaboration among multidisciplinary teams.

One of the most significant advantages of CWL is its prowess in enhancing the portability of workflows across various platforms. Take the example of a remote sensing algorithm for cloud detection. By encapsulating the algorithm's steps and dependencies within a CWL document, the workflow becomes platform-agnostic. Whether executed on a local machine, a cloud instance, or within a containerized environment, the workflow remains consistent. This cross-platform compatibility eliminates the challenges of software version disparities and configuration differences, ensuring reproducible results regardless of the execution context.

CWL encompasses two main classes that define the structure of workflows:

- **CommandLineTool:** Represents a command-line executable, complete with input parameters, command-line arguments, and output specifications. This class is essential for encapsulating standalone processing steps, such as running image processing algorithms.

- **Workflow:** Defines a sequence of connected steps, including inputs, outputs, and dependencies. Workflows allow the integration of multiple `CommandLineTools` and provide a holistic representation of complex processing pipelines.

The `CommandLineTool` class within the CWL serves as a fundamental building block for creating executable workflows. It represents a self-contained command-line executable, encapsulating the necessary details to execute a specific processing step. This class proves particularly useful for isolating individual algorithms or tasks within a larger workflow.

In the definition of a CWL `CommandLineTool`, several essential components are outlined:

- **inputs:** This section describes the input parameters required by the command-line tool. Inputs can encompass various data types such as file references, strings, integers, and more.
- **outputs:** The outputs section specifies the expected output files or values generated by the tool after execution.
- **baseCommand:** This field defines the command that needs execution, along with its initial arguments.
- **arguments:** Additional command-line arguments that are appended to the base command during execution.
- **stdout, stderr:** These fields designate the paths for standard output and standard error, facilitating the capture of messages and errors.

For instance, in the context of calculating the Normalised Difference Vegetation Index (NDVI) from input satellite imagery, the following CWL representation can be observed:

```
cwlVersion: v1.2
class: CommandLineTool
baseCommand: python calculate_ndvi.py
inputs:
  - id: input_image
    type: File
  - id: output_ndvi
    type: File
outputs:
  - id: output_ndvi
    type: File
stdout: output.log
stderr: error.log
```

The `Workflow` class within CWL empowers the creation of intricate workflows by connecting multiple tools or steps. Workflows provide a means to orchestrate the execution of `CommandLineTools`, establishing a sequence of operations aimed at achieving a specific task. Workflows can include parallel processing, conditional branching, and the management of data dependencies.

In a CWL Workflow definition, the primary components include:

- inputs: This field specifies the inputs that the workflow anticipates.
- outputs: The outputs section outlines the results generated by the workflow.
- steps: This section defines an array of steps, representing the tools or workflows being executed. Each step includes the tool's identifier and its inputs.
- requirements: Describes the requirements necessary for executing the workflow, such as software dependencies or computational resources.

Expanding upon the CommandLineTool example, consider the creation of a Workflow that pre-processes an image and subsequently calculates NDVI using distinct tools:

```
cwlVersion: v1.2
class: Workflow
inputs:
  - id: input_image
    type: File
outputs:
  - id: output_ndvi
    type: File
steps:
  preprocess:
    run: preprocess.cwl
    in:
      input_image: input_image
    out: [preprocessed_image]
  calculate_ndvi:
    run: calculate_ndvi.cwl
    in:
      input_image: preprocess/preprocessed_image
    out: [output_ndvi]
```

In this example, the Workflow interconnects two steps, 'preprocess' and 'calculate_ndvi'. Each step references a CommandLineTool definition (preprocess.cwl and calculate_ndvi.cwl) that executes distinct tasks. The output of the 'preprocess' step serves as the input for the 'calculate_ndvi' step.

The CommandLineTool and Workflow classes in CWL coalesce, offering a platform for creating structured workflows that encompass both individual processing steps and complex orchestrations. This combined approach empowers users to craft, share, and execute data processing workflows with ease, enabling execution across diverse platforms and environments.

6.3 Containers

Containerization has emerged as an approach in modern software development, transforming the way applications are packaged, deployed, and executed. In the realm of

EO, where data processing precision and reproducibility are paramount, containerization offers a powerful solution to address the challenges of varying computing environments. By encapsulating applications, dependencies, libraries, and configurations into self-contained units called containers, developers ensure consistent and predictable execution across diverse platforms.

For instance, consider an EO Application Package aimed at analysing forest cover using satellite imagery. The package involves multiple processing steps, such as data preprocessing, feature extraction, and classification. By containerizing each processing step, the researcher creates isolated and standardised environments. These containers not only house the necessary algorithms but also encompass the specific libraries and tools required for accurate analysis. When the EO Application Package is shared or deployed on different systems, the containerization approach guarantees that the analysis remains reliable and unaffected by variations in the underlying computing environment.

The integration of containerization with the CWL further amplifies the benefits for EO Application Packaging. CWL provides a standardised syntax to describe workflows, enabling the creation of executable workflows that are independent of the execution platform. By referencing container images within CWL workflow definitions, developers ensure that the same execution environment is consistently maintained, regardless of where the workflow is executed.

A pivotal facet of containerization lies in effective dependency management, ensuring that the application operates harmoniously within its encapsulated environment. Dependency management tackles the challenge of varying software libraries and configurations across different computing systems. Containers encapsulate not only the application itself but also its dependencies, liberating developers from compatibility struggles.

Consider an EO Application Package that detects land cover changes using a combination of spectral indices from satellite data. Such an application necessitates specialised libraries for data manipulation, image processing, and spectral analysis. By incorporating these dependencies into the container image, developers can ensure that the application functions across diverse execution environments. Dependency management within containers safeguards against version conflicts, system-specific adjustments, and other potential pitfalls that might compromise the accuracy of EO data processing.

6.4 Deployment

Once the application package is stored in Harbor, it can be deployed within the CopernicusLAC platform. The deployment process involves:

- **Preparing Deployment Configurations:** Managed through Helm charts, these configurations define how the application should be deployed in Kubernetes, including resource allocation, environment variables, and network settings.

- Deploying to Kubernetes: The application is deployed as Kubernetes pods, with each pod running a containerized version of the application.
- Monitoring and Scaling: Kubernetes monitors the application to ensure it operates correctly and can scale resources up or down based on demand.

Nevertheless the Application Package is designed to support a wide range of deployment scenarios, ensuring that EO applications can be executed across various environments. This flexibility is crucial for adapting to different computational needs, from local development to large-scale cloud deployments. The platform leverages the standardised packaging process, primarily based on CWL, to ensure that applications are portable and can be deployed consistently across these environments.

6.4.1 Local Computer Execution

- Purpose: Local execution is typically used during the development and testing phases. Developers can run applications on their local machines to debug and refine workflows before deploying them in more complex environments.
- Implementation: The CWL-backed application package simplifies deployment by encapsulating all necessary tools, configurations, and dependencies. This ensures that the application behaves consistently across different local setups, eliminating environment-specific issues.
- Advantages: Fast feedback loops, easy debugging, and iterative development.

6.4.2 Kubernetes Cluster Execution

- Purpose: Kubernetes is widely used for managing containerized applications in a scalable and resilient manner. This scenario is ideal for applications that need to be distributed across multiple nodes, ensuring high availability and fault tolerance.
- Implementation: The CWL-driven application package is well-suited for Kubernetes environments. It includes details on container specifications, resource allocations, and inter-container communications, allowing Kubernetes to manage the deployment and scaling of the application effectively.
- Advantages: Automatic scaling, load balancing, and management of complex, distributed applications.

6.4.3 Execution as a Service

- Purpose: In this scenario, applications are deployed as services that can be accessed via APIs, particularly through the OGC API Processes interface. This approach makes EO applications available as web services, enabling integration with other geospatial services and tools.
- Implementation: The CWL blueprint ensures that the application package is compatible with OGC standards, facilitating deployment as a standardised service.

This allows other users and systems to interact with the application through a well-defined API, enhancing interoperability and usability.

- Advantages: Broad accessibility, interoperability with other platforms, and the ability to offer EO applications as a service to a wide audience.

6.5 Version Control and Update Mechanisms

Version control is a key aspect of the application packaging process, ensuring that EO applications are maintained consistently and can be reliably reproduced. In the platform, GitLab serves as the central tool for managing version control and automating the update process.

The key components are:

- **GitLab Repositories:** All application code, workflows, and Dockerfiles are stored in GitLab repositories. Each repository serves as a centralised space where developers can manage their application's source code, configuration files, and documentation. This setup ensures that all aspects of the application are versioned and can be tracked over time.
- **Branching and Merging:** GitLab supports branching and merging workflows, enabling multiple developers to work simultaneously on different features or fixes. Changes can be isolated in branches, tested independently, and then merged back into the main branch when ready. This workflow minimises the risk of conflicts and ensures that the main codebase remains stable.
- **Tagging and Releases:** GitLab's tagging feature is used to mark specific points in the application's history, such as version 1.0 or a particular release candidate. These tags allow for easy retrieval and deployment of specific versions of the application, providing a clear history of changes and ensuring that older versions can be redeployed if necessary.
- **Continuous Integration (CI):** GitLab's CI/CD pipelines automate the process of integrating code changes, running tests, and deploying updated packages. When a developer pushes changes to a GitLab repository, the CI pipeline is triggered, executing a series of predefined tasks to ensure that the application is built correctly, tested for regressions, and packaged into a Docker image.

An example of the version control and update mechanism is:

1. **Code Commit:** A developer commits changes to the repository, which might include updates to the application code or configuration files.
2. **CI Pipeline Execution:** GitLab's CI pipeline is automatically triggered. It begins by pulling the latest code, running a series of unit and integration tests, and then building a new Docker image if all tests pass.
3. **Testing and Validation:** The new Docker image undergoes testing to ensure that it functions as expected. This includes running the application in a staging environment to simulate real-world conditions.

4. Deployment: If the image passes all tests, it is automatically pushed to Harbor and tagged with a new version number. The Helm charts are updated to reflect the new image version, and Kubernetes handles the deployment.
5. Rollback Mechanism: In the event of a failure or issue with the new version, the platform's version control system allows for quick rollbacks to a previous stable version, minimising downtime and disruption

Reproducibility is one of the most significant advantages offered by the CopernicusLAC platform's approach to application packaging and deployment. By maintaining strict version control through GitLab, the platform ensures that applications can be reliably reproduced across different environments and over time.

Collaboration is greatly enhanced by GitLab's set of tools for branching, merging, and project management. These tools allow multiple developers to work on the same project simultaneously, without the risk of conflicts or code overwrites. Developers can create branches to experiment with new features or fixes, merge changes back into the main codebase once they are tested and approved, and collaborate across distributed teams.

Automation plays a role in the efficiency and reliability of the CopernicusLAC platform. The use of Continuous Integration and Continuous Deployment (CI/CD) pipelines automates the processes of building, testing, and deploying EO applications. This automation minimises the risk of human error, as tasks that were once manual and prone to mistakes are now consistently executed by automated scripts. Furthermore, automation speeds up the development cycle, allowing developers to quickly push updates, run tests, and deploy new versions of applications with minimal downtime. This leads to a more agile development process, where new features and improvements can be rolled out rapidly.

6.6 Access Control

Access control supports the managing of application packages by ensuring that only authorised users can access, modify, or deploy application packages is essential for maintaining the security and integrity of the platform's operations. The CopernicusLAC platform employs Keycloak as its central identity and access management system, integrating with tools like GitLab and Harbor to manage permissions and secure access to application packages.

The key components of the access control mechanism are:

- Keycloak Identity and Access Management:
 - **Centralised Authentication and Authorization:** Keycloak provides centralised authentication and authorization services for the CopernicusLAC platform. Users log in through Keycloak, which manages their identity and assigns appropriate roles and permissions across the platform's various services, including GitLab and Harbor.

- **Single Sign-On (SSO):** Keycloak enables SSO across all integrated services. This means users only need to authenticate once to access GitLab, Harbor, and other platform services, streamlining the user experience and ensuring consistent access control.
- **Role-Based Access Control (RBAC):** Keycloak allows administrators to define and manage roles centrally. These roles determine what actions a user can perform across different services. For instance, a user might have a "Developer" role in GitLab and a "Maintainer" role in Harbor, with each role granting different levels of access.
- **Federated Identity:** Keycloak supports federated identity management, allowing the integration of external identity providers (e.g., LDAP, Active Directory, social logins). This enables integration with existing organisational identity systems.
- **GitLab Access Controls:**
 - **Integration with Keycloak:** GitLab is integrated with Keycloak for user authentication and role management. Users' roles in GitLab are determined by the permissions assigned to them in Keycloak, ensuring consistent and secure access across the platform.
 - **Protected Branches:** GitLab allows for protected branches, where only users with specific roles (as determined by Keycloak) can push changes or merge code. This ensures that critical branches, such as the main or production branches, are safeguarded against unauthorised modifications.
- **Harbor Access Management:**
 - **User and Project Management via Keycloak:** Harbor's access control is also managed through Keycloak. This integration ensures that users' access to container images is consistent with their assigned roles, providing a secure environment for storing and deploying application packages.
 - **Vulnerability Scanning and Access Restriction:** Harbor integrates with vulnerability scanning tools and can restrict access to container images based on scan results. This ensures that only secure and validated images are available for deployment, aligned with the access controls managed by Keycloak.
- **Authentication and Authorization:**
 - **Token-Based Authentication:** Keycloak supports token-based authentication for API access and automated processes. Users and services can generate access tokens with specific scopes and expiration dates, providing secure, granular access to resources without exposing user credentials.
 - **Multi-Factor Authentication (MFA):** Keycloak enforces MFA policies, adding an additional layer of security. Users must provide a second form of verification, such as a mobile app code, when accessing sensitive parts of the platform.

The access control implementation should follow the steps below:

1. **Role Assignment in Keycloak:** Administrators define user roles in Keycloak, which are then propagated across all integrated services like GitLab and Harbor. This centralised role management simplifies access control and ensures consistency across the platform.
2. **Access Monitoring and Auditing:** Keycloak provides logging and auditing features that track user activity across the platform. This allows administrators to monitor access patterns, detect unauthorised access attempts, and audit changes to application packages.
3. **Security Policies Enforcement:** The platform leverages Keycloak's advanced security features, such as mandatory multi-factor authentication (MFA) and regular audits of access permissions, to enhance security. These policies ensure that access is granted only to authorised users and that sensitive operations are securely managed.

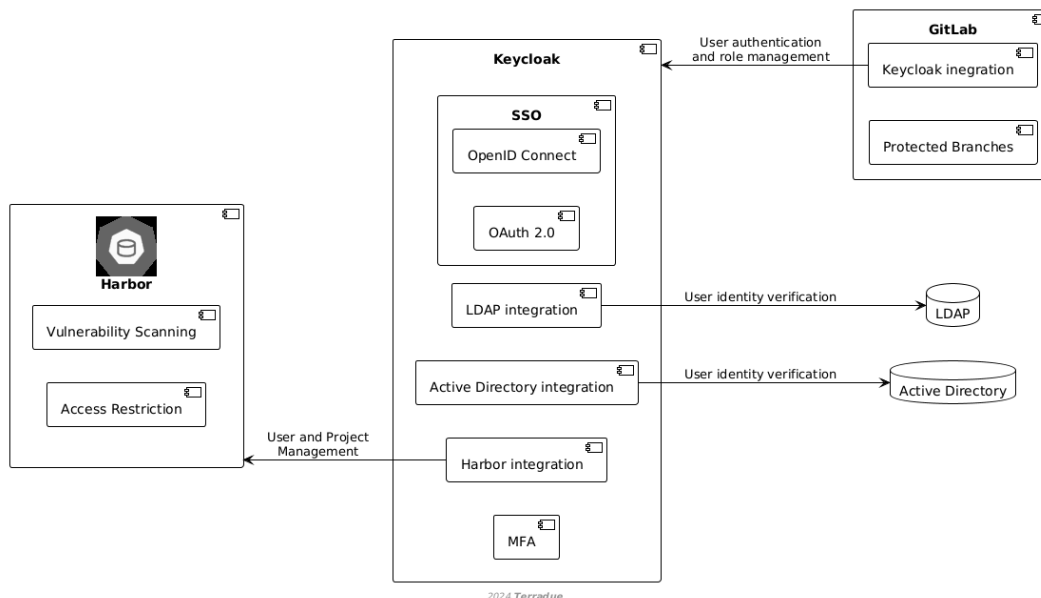


Figure 18: Access Control Architecture Diagram

The diagram above illustrates how Keycloak integrates with GitLab and Harbor to manage user authentication, role-based access control, and security features such as vulnerability scanning and multi-factor authentication. This architecture ensures secure and consistent access management across the platform's services.

6.7 Benefits

The application packaging process within the CopernicusLAC platform brings a multitude of benefits that enhance the overall efficiency, security, and scalability of EO data processing. By utilising technologies such as CWL, Docker, and container registries like Harbor, the platform ensures that applications are consistently deployed across diverse environments,

from local machines to cloud infrastructures and high-performance computing systems. This consistency is important for maintaining the integrity and reproducibility of data processing workflows, which is essential in scientific research and operational settings.

The use of containers, managed through Docker and orchestrated by Kubernetes, provides another layer of flexibility and scalability. Containers encapsulate all necessary software, dependencies, and configurations, enabling the application to run consistently across various environments. This containerization ensures that applications are not affected by differences in underlying system configurations, making them highly portable and easy to scale. When combined with Kubernetes, the platform can manage the deployment and scaling of these containers, optimising resource usage and ensuring that applications remain responsive and available even under varying loads.

Moreover, the automated version control and continuous integration (CI) pipelines provided by GitLab contribute significantly to the reliability and quality of the applications. These tools ensure that every change to the application code is systematically tracked, tested, and deployed, minimising the potential for errors and inconsistencies. The ability to automate the build, test, and deployment processes not only speeds up the development lifecycle but also ensures that the deployed applications are thoroughly validated before being released into production. This level of automation is crucial in maintaining high standards of reliability and performance, particularly in complex and large-scale EO data processing tasks.

By integrating Keycloak as the central identity and access management system, the platform achieves a unified and secure method of managing user roles and permissions across all its services, including GitLab and Harbor. This integration simplifies the administrative burden of managing access controls, ensuring that only authorised users can modify or deploy application packages. The use of role-based access control (RBAC) and multi-factor authentication (MFA) further strengthens the security posture of the platform, protecting sensitive operations from unauthorised access and reducing the risk of security breaches.

6.8 Compliance with Platform Standards

To ensure integration and reliable operation of EO applications within the CopernicusLAC platform, developers must adhere to specific platform standards during the application packaging process. Compliance with these standards is essential for maintaining consistency, security, and performance across the platform's various components. This subsection outlines the key standards and practices that developers must follow when packaging their EO algorithms.

6.8.1 Use of Common Workflow Language (CWL)

All EO applications **MUST** be described using the Common Workflow Language (CWL). This ensures that workflows are portable and reproducible, enabling consistent execution across different environments. CWL provides a standardised syntax for defining workflows,

specifying input and output parameters, and detailing the dependencies between different processing steps.

6.8.2 Unattended Execution Capability

All EO applications **MUST** be designed to run in an unattended or automated execution mode. This means that once deployed, the applications should be capable of executing their workflows, processing data, and producing outputs without the need for human intervention. This ensures that applications can be integrated into larger automated pipelines, supporting continuous operations and large-scale data processing within the CopernicusLAC platform.

6.8.3 Docker Containerization

EO applications **MUST** be encapsulated within Docker containers. These containers should include all necessary dependencies and configurations to ensure that the application runs consistently, regardless of the underlying infrastructure. Developers are encouraged to use lightweight base images and to optimise container size and performance.

6.8.4 Adherence to Metadata Standards

Applications **MUST** produce outputs that comply with the platform's metadata standards, ensuring that data is discoverable and usable within the platform. This includes adhering to the minimal metadata profile and ensuring that outputs are catalogued using the SpatioTemporal Asset Catalog (STAC) format.

6.8.5 Security and Vulnerability Management

Developers **MUST** ensure that their Docker containers are free from known vulnerabilities by conducting regular security scans. Applications should be designed to comply with the platform's security protocols.

6.8.6 GitLab for Version Control

The source code, CWL workflows, and Dockerfiles for EO applications **SHOULD** be managed using GitLab. Developers should follow best practices for version control, including tagging releases and maintaining clear documentation of changes. GitLab's CI/CD pipelines should be used to automate the build, test, and deployment processes.

6.8.7 Resource Optimization and Performance

To optimise performance, developers **SHOULD** define resource requests and limits within their Kubernetes deployment configurations. Applications should support parallel processing and efficient resource utilisation, ensuring that they can handle large datasets and high workloads without degrading platform performance.

6.8.8 Comprehensive Documentation

All EO applications **SHOULD** be accompanied by detailed documentation. This includes descriptions of the application's functionality, input/output specifications, usage examples, and deployment instructions. Documentation is crucial for facilitating the use, maintenance, and troubleshooting of applications within the platform.

6.8.9 Testing and Validation

Before deployment, applications **SHOULD** undergo thorough testing and validation. This includes unit tests, integration tests, and performance benchmarks. Developers should provide test datasets and ensure that their applications are fully validated within the platform's CI/CD pipeline.

7 Technologies and Frameworks

7.1 Purpose

The purpose of detailing the technologies and frameworks used in the CopernicusLAC platform is to provide a clear understanding of the technical foundation that supports the platform's functionality. This section aims to justify the selection of specific technologies based on their capabilities, describe how they are implemented, and explain their integration within the platform. Additionally, it enhances transparency for stakeholders, supports development and maintenance efforts, and facilitates knowledge transfer by offering detailed documentation on the technologies used. By doing so, this section ensures that the platform adheres to best practices and operates reliably, and securely.

7.2 Common Workflow Language (CWL)

The Common Workflow Language (CWL) is integral to the CopernicusLAC platform, providing a standardised approach for defining and executing complex data processing workflows. CWL's primary strength lies in its ability to describe workflows in a way that is both platform-independent and highly reproducible. This standardisation ensures that workflows can be consistently executed across a variety of environments, from local development setups to large-scale cloud deployments.

To execute CWL workflows, several tools and engines have been developed, each optimised for different environments and use cases. These tools ensure that the workflows described in CWL can be executed at scale, with support for parallel processing and integration with modern infrastructure:

- **cwltool:** cwltool is the reference implementation of CWL, developed by the CWL project itself. It is a lightweight, Python-based tool designed for testing and running CWL workflows on a local machine. While cwltool is ideal for development and small-scale execution, it lacks the scalability required for large, production-grade environments.
- **Toil:** Toil is a workflow engine designed for scalable, distributed computing environments. It supports the execution of CWL workflows across multiple compute nodes, making it suitable for high-performance computing (HPC) environments and cloud-based deployments. Toil integrates well with batch processing systems like SLURM and cloud services such as AWS Batch, allowing for flexible and powerful execution strategies.
- **Calrissian:** Calrissian is a specialised CWL implementation designed to run within a Kubernetes cluster. Its primary goal is to harness the power of Kubernetes to execute workflows, taking full advantage of high-capacity clusters to run multiple steps in parallel. Calrissian is highly scalable and can dynamically allocate resources based

on the requirements of each workflow step, making it an ideal choice for environments where large-scale data processing tasks need to be executed reliably.

Calrissian integrates with Kubernetes, allowing it to handle the orchestration of containers that execute individual workflow steps. By leveraging Kubernetes' built-in features, such as auto-scaling, load balancing, and fault tolerance, Calrissian ensures that workflows are executed with optimal resource utilisation and minimal downtime. This makes it particularly valuable for the CopernicusLAC platform, where the ability to scale workflows across a distributed infrastructure is crucial.

7.3 Docker

Docker is a containerization technology that allows the CopernicusLAC platform to encapsulate applications and their dependencies into self-contained units called containers. These containers ensure that applications run consistently across different environments by isolating them from the underlying system. Docker is pivotal in managing the diverse software requirements of EO applications, providing a standardised runtime environment that includes all necessary libraries and configurations. This approach simplifies deployment, enhances security, and ensures that the applications are easily scalable.

7.4 Kubernetes

Kubernetes is a powerful container orchestration platform used by the CopernicusLAC platform to manage the deployment, scaling, and operation of containerized applications. Kubernetes automates the deployment process, ensuring that containers are distributed across the available infrastructure, balancing the load and maximising resource utilisation. This technology is responsible for maintaining high availability and resilience in the platform, as it can automatically restart or relocate containers if they fail, ensuring uninterrupted service.

7.5 GitLab

GitLab serves as the version control system and Continuous Integration/Continuous Deployment (CI/CD) platform for the CopernicusLAC platform. GitLab is used to manage the source code of EO applications, track changes over time, and facilitate collaboration among developers. The CI/CD pipelines in GitLab automate the process of building, testing, and deploying applications, ensuring that updates are deployed quickly and reliably. This integration streamlines development workflows and helps maintain the integrity and consistency of the platform's applications.

7.6 Harbor

Harbor is an open-source container image registry used by the CopernicusLAC platform to store and manage Docker images securely. Harbor provides advanced security features, such as vulnerability scanning, image signing, and access control, ensuring that only trusted and verified images are deployed on the platform. By organising images into projects and enforcing role-based access control (RBAC), Harbor allows for the management of containerized applications, contributing to the overall security and reliability of the platform.

7.7 Argo Workflows

Argo Workflows is an open-source container-native workflow engine for orchestrating parallel jobs on Kubernetes. Within the CopernicusLAC platform, Argo Workflows is used to define, schedule, and execute complex workflows that involve multiple steps and dependencies. By leveraging Kubernetes' scalability and Argo's workflow management capabilities, the platform can process large EO datasets, automate repetitive tasks, and ensure that workflows are executed reliably in a distributed environment.

7.8 Kafka

Apache Kafka is a distributed event streaming platform used for building real-time data pipelines and streaming applications. In the CopernicusLAC platform, Kafka is employed to manage and process streams of data generated by EO sensors and processing workflows. Kafka's ability to handle high-throughput, low-latency data streams makes it an essential component for managing the real-time aspects of EO data ingestion, processing, and analysis. By integrating Kafka, the platform can ensure that data flows smoothly between components, enabling real-time analytics and decision-making.

7.9 Crossplane

Crossplane is an open-source control plane that extends Kubernetes to manage cloud-based infrastructure and services. In the CopernicusLAC platform, Crossplane is used to provision and manage cloud resources dynamically, such as databases, storage, and networking components, as part of the application deployment process. Crossplane's ability to treat infrastructure as code allows the platform to automate the management of its cloud environment, ensuring that resources are allocated and that the infrastructure scales in response to demand.

7.10 Prometheus

Prometheus is an open-source monitoring and alerting toolkit designed for reliability and scalability. In the CopernicusLAC platform, Prometheus is used to collect and store metrics

from various components of the system, providing insights into the platform's performance and health. Prometheus scrapes metrics from configured endpoints, stores them, and allows for real-time querying and analysis.

Prometheus is expected to be deployed within the platform's Kubernetes environment, leveraging its ability to monitor containerized applications at scale. This setup ensures that the platform can continuously monitor key metrics such as CPU usage, memory consumption, and network activity, enabling proactive detection of issues and resource management. By integrating Prometheus, the platform can maintain high availability and performance, quickly identifying and resolving potential bottlenecks or failures.

7.11 Grafana

Grafana is an open-source analytics and monitoring platform that is used in conjunction with Prometheus to visualise metrics and data collected from the CopernicusLAC platform. Grafana provides a powerful and flexible interface for creating and sharing dashboards, enabling users to visualise system performance, track the health of applications, and analyse trends over time.

In the CopernicusLAC platform, Grafana is expected to be deployed as part of the monitoring stack, with integration with Prometheus for real-time data visualisation. This setup allows administrators and developers to create custom dashboards that display relevant metrics, such as system load, response times, and error rates, in an intuitive and interactive way. By using Grafana, the platform ensures that its operational data is accessible and actionable, supporting informed decision-making and timely responses to any performance issues.

7.12 ZOO Project

The ZOO Project is an open-source platform that provides a framework for implementing and deploying Web Processing Services (WPS). In the CopernicusLAC platform, ZOO Project is utilised to handle geospatial processing tasks, allowing EO applications to be exposed as web services that can be accessed and executed via standard protocols. This capability enhances the platform's interoperability and enables users to integrate geospatial processing into their workflows.

7.13 Titiler

Titiler is a dynamic tiling service that generates map tiles from geospatial data, allowing for real-time visualisation of EO datasets within the CopernicusLAC platform. Titiler is particularly well-suited for handling COGs, enabling users to interact with large datasets by streaming only the necessary portions of the data. This technology enhances the platform's

visualisation capabilities, providing users with the ability to explore and analyse geospatial data through interactive maps.

7.14 PostgreSQL

PostgreSQL is a powerful, open-source relational database management system (RDBMS) utilised by the CopernicusLAC platform for storing and managing metadata, particularly in relation to the SpatioTemporal Asset Catalog (STAC). PostgreSQL is known for its robustness, scalability, and support for complex queries, making it an ideal choice for handling the large volumes of metadata generated by EO data processing tasks.

In the context of the CopernicusLAC platform, PostgreSQL is expected to be provided as a managed service by the cloud provider. This approach offers several advantages, including automated maintenance, backups, and scaling, allowing the platform to focus on its core functionalities rather than database management. By leveraging PostgreSQL as a service, the platform benefits from the cloud provider's expertise in ensuring high availability, security, and performance, further enhancing the reliability and efficiency of metadata storage and retrieval.

8 Security and Compliance

8.1 Purpose

The purpose of this section is to outline the security and compliance objectives within the CopernicusLAC platform. Ensuring the protection of data, particularly personal data, and adhering to regulatory requirements is key to maintaining the integrity, trust, and operational effectiveness of the platform. This section will detail the strategies and practices employed to safeguard data, manage access, and respond to security incidents, all while complying with relevant legal and organisational standards, including the ESA Personal Data Protection Framework (PDP).

8.2 Security Framework

The CopernicusLAC platform implements a comprehensive security framework designed to protect data and ensure compliance with ESA and international standards. The framework includes a combination of methodologies and protocols that govern data handling, transmission, and storage. These measures are aligned with best practices for cybersecurity, ensuring that the platform can defend against threats and vulnerabilities.

Key components of the security framework include:

- **Data Encryption:** All sensitive data is encrypted both at rest and in transit using industry-standard encryption protocols (e.g., TLS 1.2 or later). This ensures that data remains secure during transmission across networks and while stored within the platform's infrastructure.
- **Access Control:** Role-Based Access Control (RBAC) mechanisms are in place to ensure that only authorised personnel have access to sensitive data and systems. This minimises the risk of unauthorised access or data breaches.
- **Secure Data Transfer:** The platform employs secure data transfer protocols, such as HTTPS and SFTP, to protect data during transmission. These protocols ensure that data is encrypted and secure from interception or unauthorised access during transfer.
- **Regular Security Audits:** The platform undergoes regular security audits to identify potential vulnerabilities and ensure that security measures are up-to-date and effective.

8.3 Data Protection and Privacy

Data protection and privacy are of paramount importance in the CopernicusLAC platform, particularly given its compliance with the ESA Personal Data Protection Framework (PDP). The platform adheres to stringent data protection principles, including data minimization, purpose limitation, and ensuring the rights of data subjects:

- **ESA PDP Compliance:** The platform complies with the ESA PDP Framework, which governs the protection of personal data within ESA's operations. This includes adherence to the EU Standard Contractual Clauses for the Transfer of Personal Data to third countries, ensuring that any data transfers outside the EU are authorised by the ESA Data Protection Officer (DPO) and subject to adequate safeguards.
- **Data Minimization:** Only the minimum amount of personal data necessary for the operation of the platform is collected and processed. This principle helps to reduce the risk associated with data breaches and ensures compliance with privacy regulations.
- **User Consent Management:** The platform includes mechanisms for obtaining and managing user consent for data processing activities. Users are informed about how their data will be used, and they have the ability to withdraw consent at any time.
- **Data Subject Rights:** The platform is designed to facilitate the exercise of data subject rights, such as access, rectification, erasure, and data portability, in compliance with applicable data protection laws.

8.4 Access Controls

Access control is a core component of the platform's security strategy, ensuring that only authorised individuals can access sensitive data and systems. It includes:

- **Role-Based Access Control (RBAC):** The platform employs RBAC to manage user access based on roles and responsibilities. This system restricts access to data and functionalities based on the principle of least privilege, ensuring that users only have access to the information and tools necessary for their roles.
- **Multi-Factor Authentication (MFA):** MFA is implemented to add an additional layer of security by requiring users to provide multiple forms of verification before gaining access.
- **Audit Logging:** All access attempts and actions taken by users are logged and monitored to detect and respond to unauthorised access attempts. These logs are regularly reviewed as part of the platform's ongoing security monitoring efforts.

The use of Keycloak in the CopernicusLAC platform brings several significant benefits, foremost of which is the unification of access management. By centralising identity and access management through Keycloak, the platform simplifies the administration of user roles and permissions across multiple integrated services. This unification streamlines the process of managing access controls, reducing complexity for administrators and ensuring that permissions are consistently applied throughout the platform. This centralised approach not only enhances operational efficiency but also helps maintain a coherent security posture across different components of the platform.

Keycloak also significantly enhances the security of the CopernicusLAC platform. Its authentication and authorization mechanisms ensure that only authorised users can access sensitive resources. The integration of multi-factor authentication (MFA) adds an additional

layer of security, further reducing the risk of unauthorised access. By requiring multiple forms of verification, MFA ensures that even if user credentials are compromised, unauthorised access is still unlikely. This comprehensive security framework helps safeguard the integrity of application packages and other resources, providing confidence in the platform's overall security.

Additionally, Keycloak provides scalability and flexibility, allowing the CopernicusLAC platform to grow securely as the number of users and services increases. Keycloak's support for federated identity and role-based access control ensures that the platform can accommodate a growing user base while maintaining stringent security standards. This flexibility is particularly important in environments where users may need access from various identity providers, or where roles and permissions need to be dynamically managed. Keycloak's capabilities ensure that as the platform expands, it can continue to enforce consistent and secure access controls, adapting to evolving security needs without compromising performance or usability.

8.5 Monitoring and Auditing

The CopernicusLAC platform employs monitoring and auditing practices to maintain security and ensure compliance with regulatory requirements:

- **Continuous Monitoring:** The platform uses tools like Prometheus for continuous monitoring of system performance and security metrics. This allows for real-time detection of anomalies or potential security threats.
- **Security Audits:** Regular security audits are conducted to assess the effectiveness of the platform's security measures. These audits help identify potential vulnerabilities and ensure that security protocols are being followed.
- **Compliance Audits:** Compliance with ESA's data protection framework and other relevant regulations is regularly reviewed to ensure ongoing adherence. These audits involve checking that all data handling practices meet the required standards and that any issues are promptly addressed.

8.6 Incident Response

The platform has a comprehensive incident response plan to manage and mitigate the impact of security incidents:

- **Incident Detection:** The platform uses automated monitoring tools to detect potential security incidents in real-time. These tools are configured to alert the security team immediately when anomalies or breaches are detected.
- **Containment and Mitigation:** Upon detection of an incident, the security team follows predefined procedures to contain the breach and mitigate its impact. This may involve isolating affected systems, revoking access privileges, or deploying security patches.

- **Recovery:** The platform has protocols in place for restoring normal operations following an incident. This includes data recovery processes, system restoration from backups, and thorough testing to ensure that the issue has been resolved.
- **Notification and Reporting:** In compliance with the ESA PDP Framework, any incidents involving personal data are reported to the ESA DPO and, where required, to the affected data subjects. Detailed incident reports are also prepared to document the event, the response actions taken, and any lessons learned.

9 Scalability and Performance

9.1 Purpose

The purpose of this section is to define the goals and objectives related to scalability and performance within the CopernicusLAC platform. Ensuring that the platform can scale to handle increasing workloads and maintain optimal performance under varying conditions is critical to its success. This section outlines the strategies, optimizations, and processes implemented to achieve these goals, ensuring that the platform can support growing user demands and complex data processing tasks without compromising on speed, reliability, or resource efficiency.

9.2 Scalability Strategies

The CopernicusLAC platform employs several strategies to ensure it can scale effectively to meet increasing demands:

- **Containerization with Kubernetes:** The platform uses Kubernetes to manage containerized applications, enabling dynamic scaling based on workload demands. Kubernetes automatically adjusts the number of running containers, scaling up to handle high loads and scaling down during periods of lower demand, ensuring efficient use of resources.
- **Microservices Architecture:** By adopting a microservices architecture, the platform can scale individual services independently. This approach allows for targeted scaling, where only the services experiencing high demand are scaled, reducing overhead and improving performance.
- **Cloud-Native Technologies:** The platform leverages cloud-native technologies like Crossplane to provision and manage cloud resources dynamically. Crossplane enables the platform to scale infrastructure components such as databases, storage, and compute resources in response to increasing demands, ensuring that the platform remains responsive and performant.
- **Distributed Data Management:** The use of distributed databases like PostgreSQL and data storage solutions ensures that data can be accessed and processed, even as the volume of data grows. This approach helps in maintaining high availability and quick response times as the platform scales.

9.3 Performance Optimization

Performance optimization is a key focus for the CopernicusLAC platform, ensuring that it operates across all components:

- **Load Balancing:** The platform employs load balancers to distribute incoming traffic evenly across multiple servers or containers. This prevents any single component

from becoming a bottleneck and helps maintain optimal performance during peak usage times.

- **Caching Strategies:** The platform uses caching mechanisms to store frequently accessed data in memory, reducing the need for repeated database queries or data retrieval operations. This significantly improves response times and reduces the load on backend services.
- **Optimised Data Processing:** The platform optimises data processing workflows by using parallel processing and efficient algorithms, particularly in components like Argo Workflows. This ensures that large datasets can be processed quickly and effectively, minimising delays and improving overall performance.
- **Resource Allocation:** By carefully managing resource allocation using tools like Kubernetes and Crossplane, the platform ensures that each service has the necessary resources to operate, without over-provisioning or under-utilising resources.

9.4 Load Testing and Benchmarking

To ensure that the CopernicusLAC platform meets its performance requirements, rigorous load testing and benchmarking are conducted:

- **Load Testing:** The platform undergoes regular load testing to simulate high-traffic scenarios and assess how well it handles increased demand. These tests help identify potential bottlenecks and ensure that the platform can maintain performance under stress.
- **Benchmarking:** Benchmarking is used to measure the platform's performance against predefined metrics, such as response times, throughput, and resource utilisation. These benchmarks provide a baseline for performance and are used to track improvements or identify areas that need optimization.
- **Continuous Monitoring:** The platform integrates monitoring tools like Prometheus to continuously track performance metrics. This allows for real-time analysis of system performance and early detection of potential issues, enabling proactive optimization efforts.

9.5 Resource Management

The resource management supports both scalability and performance on the platform providing services for:

- **Dynamic Resource Provisioning:** The platform uses Crossplane to dynamically provision and manage cloud resources. This ensures that the platform can automatically allocate additional resources when needed, such as increasing storage capacity or adding more compute power during high-demand periods.
- **Cost-Efficient Scaling:** By optimising resource allocation, the platform ensures that it scales in a cost-efficient manner. This involves balancing the need for additional

resources with the cost implications, ensuring that the platform remains both performant and economical.

- **Automated Resource Monitoring:** The platform continuously monitors resource usage to ensure optimal performance. Tools like Prometheus and Grafana provide insights into resource consumption, helping administrators make informed decisions about scaling and resource allocation.